

Tabel 2.1 Penelitian Terdahulu

No.	Penulis	Tahun	Metode	Keterangan
1	Yanjun Gao, Ruizhe Li, John Caskey, Dmitriy Dligach, Timothy Miller, Matthew M. Churpek, Majid Afshar	2023	Model <i>DR.KNOWS</i> berbasis <i>Medical Knowledge Graph</i> untuk meningkatkan prediksi diagnosis menggunakan LLM (<i>T5-Large, GPT-3.5-turbo</i>).	Data yang digunakan berasal dari laporan kesehatan pasien. Model DR.KNOWS meningkatkan CUI Recall pada prediksi diagnosis hingga 44.58% pada dataset IN-HOUSE (dibandingkan metode berbasis ekstraksi konsep yang hanya mencapai 12.38%). Dalam tugas ringkasan diagnosis, Vanilla-T5 dengan bantuan DR.KNOWS mencapai skor tertinggi ROUGE-L 30.72 dan CUI F-score 27.78, lebih baik dibanding model tanpa KG.
2	Mingyu Jin, Qinkai Yu, Dong Shu, Chong Zhang, Lizhou Fan, Wenyue Hua, Suiyuan Zhu, Yanda Meng, Zhenting Wang, Mengnan Du, Yongfeng Zhang	2024	<i>Large Language Models</i> (LLMs), <i>Retrieval Augmented Generation</i> (RAG), XGBoost, Llama Index	Model <i>Health-LLM</i> mencapai akurasi 83.3% dan F1-score 76.2% , lebih baik dibanding <i>GPT-4 with retrieval</i> (68% accuracy, 71.8% F1-score) dan <i>fine-tuned LLaMA-2-13B</i> (73% accuracy, 67.1% F1-score). Model ini menggunakan RAG untuk meningkatkan pemahaman gejala penyakit dan <i>Llama Index</i> untuk memberikan skor keparahan gejala. Pendekatan ini memungkinkan sistem untuk memberikan prediksi penyakit yang lebih akurat dan rekomendasi kesehatan yang lebih personal.

Tabel 2.1 Penelitian Terdahulu (Lanjutan)

No.	Penulis	Tahun	Metode	Keterangan
3	Yichun Feng, Lu Zhou, Yikai Zheng, Ruikun He, Chao Ma, Yixue Li	2023	<i>Knowledge Graphs (KGs), Large Language Models (LLMs), Knowledge Graph Question Answering (KGQA)</i>	Mengintegrasikan LLM dengan knowledge graph thought dalam menjawab pertanyaan biomedis terkait kanker. Dalam pengujian menggunakan dataset Pan-cancer Question Answering (PcQA) yang terdiri dari 405 entri data, sistem mencapai akurasi 92.4% pada data uji, dengan skor BERTScore 97.7% dan skor ROUGE F1 86.8%, melampaui performa metode baseline seperti KG-GPT.
4	Karthik Soman, Peter W Rose, John H Morris, Rabia E Akbas, Brett Smith, Braian Pectoom, Catalina Villouta-Reyes, Gabriel Cerono, Yongmei Shi, Angela Rizk-Jackson, Sharat Israni, Charlotte A Nelson, Sui Huang, Sergio E Baranzini	2023	<i>Knowledge Graph-based Retrieval Augmented Generation (KG-RAG) menggunakan SPOKE KG dan LLM (Llama-2-13b, GPT-3.5-Turbo, GPT-4)</i>	Memanfaatkan Knowledge Graph SPOKE yang mengintegrasikan lebih dari 40 sumber pengetahuan biomedis untuk ekstraksi konteks yang lebih efisien. KG-RAG meningkatkan efisiensi token hingga 50% dibanding metode RAG berbasis Cypher, meningkatkan akurasi hingga 71% pada dataset MCQ untuk Llama-2, serta menghasilkan respons berbasis bukti dalam domain biomedis.
5	Mahyar Abbasian, M.Sc., Iman Azimi, Ph.D., Amir M. Rahmani, Ph.D., Ramesh Jain, Ph.D.	2024	<i>Large Language Models (LLMs), Open-source framework (openCHA),</i>	openCHA meningkatkan personalisasi respons kesehatan dengan menggabungkan data pasien dari EHR, wearable devices, dan sumber eksternal. Dalam estimasi

Tabel 2.1 Penelitian Terdahulu (Lanjutan)

No.	Penulis	Tahun	Metode	Keterangan
			<i>Multi-modal data analysis, Orchestrator for task execution and planning</i>	stres berbasis HRV, model mencapai akurasi 86%. Framework ini unggul dalam interpretabilitas tetapi menghadapi tantangan latensi akibat proses retrieval dan eksekusi multi-langkah.
6	Lameck Mbangula Amugongo, Pietro Mascheroni, Steven Geoffrey Brooks, Stefan Doering, Jan Seidel	2024	<i>Retrieval Augmented Generation (RAG), Systematic literature review, Evaluation frameworks for RAG applications</i>	RAG meningkatkan akurasi multi-choice QA pada GPT-4 dari 73.44% menjadi 79.97%, serta pada LLaMA-2-70B dari 42.20% menjadi 50.40% dalam tugas berbasis PubMed. Pada prediksi penyakit dengan XGBoost, RAG mencapai akurasi 83.3% dan F1-score 76.2%, melampaui GPT-4 with retrieval (68% accuracy, 71.8% F1-score). Studi ini menemukan bahwa RAG mengurangi hallucination dan meningkatkan transparansi, tetapi menghadapi tantangan dalam kualitas retrieval, kurangnya standar evaluasi, serta minimnya pembahasan terkait bias dan privasi.
7	Savvas Varitimiadis, Konstantinos Kotis, Dimitra Pittou, Georgios Konstantakis	2021	<i>Graph-Based Conversational AI untuk sistem multi-chatbot di museum, guna meningkatkan interaksi chatbot.</i>	Model multi-chatbot berbasis KG memungkinkan distribusi pengetahuan antar chatbot untuk meningkatkan respons yang lebih akurat dan berbasis bukti. Framework ini memungkinkan chatbot bekerja secara kolaboratif dengan berbagai sumber data tanpa bergantung pada satu repositori

Tabel 2.1 Penelitian Terdahulu (Lanjutan)

No.	Penulis	Tahun	Metode	Keterangan
				pusat. Evaluasi menunjukkan peningkatan keterlibatan pengunjung dibandingkan chatbot museum konvensional, namun tantangan utama adalah kompleksitas integrasi KG yang membutuhkan infrastruktur yang lebih canggih.
8	Siyuan Chen, Mengyue Wu, Kenny Q. Zhu, Kunyao Lan, Zhiling Zhang, Lyuchun Cui	2023	<i>LLM, Conversational Agents Chatbots, Human-Centered Design Methodology, Iterative Prompt Engineering</i>	Chatbot psikiater dengan desain prompt yang mencakup empati dan pertanyaan mendalam menunjukkan peningkatan skor Empathy (3.43) dan Expertise (3.71) dibandingkan versi tanpa empati. Evaluasi otomatis menunjukkan chatbot dengan pertanyaan mendalam mencapai akurasi diagnosis 55.56% dan symptom precision 92.24%, tetapi masih memiliki keterbatasan dalam symptom recall (38.10%). Chatbot pasien yang meniru ekspresi emosional dan resistensi menunjukkan peningkatan skor realism dari 1.93 ke 2.21, tetapi lebih sering melaporkan gejala yang salah (wrong symptom ratio 18.38%). Studi ini menunjukkan efektivitas ChatGPT dalam simulasi psikiatri tetapi masih menghadapi tantangan dalam keterandalan diagnosis dan ekspresi pasien yang lebih realistis.
9	Simone Kresevic, Mauro Giuffrè, Milos	2024	<i>LLM, Retrieval Augmented</i>	Mengembangkan kerangka kerja berbasis RAG untuk meningkatkan

Tabel 2.1 Penelitian Terdahulu (Lanjutan)

No.	Penulis	Tahun	Metode	Keterangan
	Ajcevic, Agostino Accardo, Lory S. Crocè, Dennis L. Shung		<i>Generation (RAG), Prompt Engineering, dan Ablation Study</i>	interpretasi pedoman klinis hepatologi oleh LLM. Mengonversi pedoman klinis menjadi format terstruktur yang dapat diproses secara efisien oleh LLM, dengan fokus pada manajemen infeksi Hepatitis C. Dalam pengujian menggunakan dataset yang mencakup pedoman klinis, model yang dikembangkan mencapai akurasi 99% pada data uji, menunjukkan peningkatan signifikan dari 43% pada model dasar GPT-4 Turbo. Hasil penelitian ini menyoroti pentingnya pemformatan pedoman yang terstruktur dan teknik prompt yang canggih dalam meningkatkan efektivitas integrasi LLM ke dalam sistem dukungan keputusan klinis (CDSS) untuk pengiriman pedoman yang lebih akurat dan relevan dalam konteks manajemen Hepatitis C.
10	Hanyao Huang, Ou Zheng, Dongdong Wang, Jiayi Yin, Zijin Wang, Shengxuan Ding, Heng Yin, Chuan Xu, Renjie Yang, Qian Zheng, Bing Shi	2023	<i>Multi-Modal Large Language Models (LLMs), Automated Dental Diagnosis, Cross-Modal Dental Diagnosis, Natural Language Processing (NLP)</i>	Studi menunjukkan kemampuan LLM dalam menganalisis EHR, merencanakan perawatan, dan menghasilkan dokumentasi klinis secara otomatis. Penggunaan visi- bahasa (vision-language models) dapat meningkatkan akurasi identifikasi kondisi gigi dalam radiografi, sementara integrasi analisis suara dapat membantu dalam deteksi gangguan bicara terkait

Tabel 2.1 Penelitian Terdahulu (Lanjutan)

No.	Penulis	Tahun	Metode	Keterangan
				kesehatan mulut. Tantangan utama mencakup privasi data, bias model, serta keterbatasan dalam interpretasi data multimodal kompleks.
11	Jin Ge, MD, MBA; Steve Sun; Joseph Owens, PhD; Victor Galvez; Oksana Gologorskaya, MS; Jennifer C. Lai, MD, MBA; Mark J. Pletcher, MD, MPH; Ki Lai, MS	2023	<i>RAG, LLM, Protected Health Information (PHI) Compliance, dan Data Extraction and Analysis</i>	LiVersa menjawab 10 dari 10 pertanyaan benar dalam format "ya/tidak", tetapi hanya 7 dari 10 respons dalam format penjelasan yang sepenuhnya akurat. RAG meningkatkan spesifisitas respons terkait hepatologi, tetapi keterbatasan dokumen yang digunakan menyebabkan beberapa kesalahan justifikasi. Studi ini menunjukkan bahwa RAG dapat mengurangi hallucination dalam aplikasi klinis, tetapi masih menghadapi tantangan dalam cakupan literatur dan bias konteks.
12	Julien Delile, Srayanta Mukherjee, Anton Van Pamel, Leonid Zhukov	2024	<i>Graph-Based Information Retrieval, Knowledge Graphs (KGs), RAG, dan Text Embedding Similarity</i>	Metode KG-RAG meningkatkan precision hingga 12% (vs. 5% pada ES IR) dan recall hingga 43% (vs. 17% pada ES IR) dalam pencarian target obat untuk berbagai penyakit. Pendekatan hybrid KG+ES menghasilkan precision dan recall sekitar dua kali lipat lebih baik dibanding metode individu. Studi ini menunjukkan bahwa KG dapat menangkap informasi long-tail dalam literatur biomedis yang sering terlewat oleh metode embedding similarity. Tantangan utama adalah

Tabel 2.1 Penelitian Terdahulu (Lanjutan)

No.	Penulis	Tahun	Metode	Keterangan
				kompleksitas pemetaan entitas dan keterbatasan retrieval untuk konsep yang kurang terstruktur dalam grafik.
13	Rajesh Bhayana, MD	2024	<i>LLM, Transformer Neural Networks, Multimodal Learning, and Prompt Engineering</i>	LLM, seperti GPT-4, menunjukkan kinerja dengan skor di atas 90% pada Ujian Lisensi Medis Amerika Serikat (USMLE) dan 88% pada Ujian Penerimaan Sekolah Hukum (LSAT), serta berhasil melewati ujian berbasis teks yang menyerupai ujian dewan radiologi, menunjukkan kemampuan reasoning yang lebih baik dibandingkan GPT-3.5. Namun, LLM juga memiliki keterbatasan, termasuk kecenderungan untuk menghasilkan informasi yang tidak akurat (hallucinations), kesulitan dalam penalaran kompleks, dan potensi bias dalam data pelatihan, sehingga menekankan pentingnya validasi lebih lanjut sebelum integrasi ke dalam praktik klinis.
14	Liang Yao, Jiazhen Peng, Chengsheng Mao, Yuan Luo	2024	<i>LLM, Knowledge Graph Completion, Triple Classification, Relation Prediction, and Instruction Tuning</i>	KG-LLM mencapai akurasi tertinggi dalam triple classification (96.6% pada WN11, 90.7% pada FB13), melampaui model ChatGPT dan GPT-4 pada beberapa tugas. Model ini juga unggul dalam relation prediction dengan skor Hits@1 0.71 pada YAGO3-10 dibandingkan GPT-4 (0.56). Fine-tuning model terbuka seperti LLaMA

Tabel 2.1 Penelitian Terdahulu (Lanjutan)

No.	Penulis	Tahun	Metode	Keterangan
				menghasilkan performa lebih baik dibandingkan LLM tertutup.
15	Royyan Abdurrohman	2024	<i>RAG, Model GPT-4, dan Evaluasi Kinerja (Akurasi, Presisi, Recall, F1-Score)</i>	Menguji performa chatbot yang dikembangkan menggunakan teknik RAG dan model GPT-4 untuk menjawab pertanyaan terkait taharah berdasarkan empat imam mazhab fikih. Hasil pengujian menunjukkan bahwa chatbot mencapai akurasi 90%, presisi 86.67%, recall 100%, dan F1-score 92.86%, menandakan kinerja yang sangat baik dalam memberikan jawaban yang relevan dan akurat. Chatbot ini dirancang untuk mengurangi halusinasi yang sering terjadi pada LLM dengan memanfaatkan basis data graf sebagai sumber informasi tambahan.

2.11 Perbedaan Penelitian

Penelitian ini mengusung sejumlah pendekatan baru dibandingkan dengan studi-studi sebelumnya, terutama melalui integrasi unik antara pendekatan *Retrieval-Augmented Generation* (RAG) dan basis data graf dalam arsitektur GraphRAG. Metode gabungan ini berbeda dari penelitian terdahulu yang cenderung hanya menggunakan salah satu dari kedua pendekatan tersebut (Ge dkk., 2023; Delile dkk., 2024). Keunikan ini didukung oleh penggunaan dataset klinis yang komprehensif, MIMIC-IV v3.1, berbeda dengan penelitian sebelumnya yang berfokus pada data yang lebih spesifik seperti pedoman penyakit hati (Ge dkk., 2023) atau literatur biomedis umum (Delile dkk., 2024). Dari sisi evaluasi, penelitian ini mengimplementasikan framework RAGAS (*Retrieval Augmented Generation Assessment System*) yang mengukur kinerja

secara mendetail melalui atribut seperti *faithfulness*, *context relevance*, *context precision*, dan *context recall*. Lebih lanjut, penelitian ini memperluas cakupan inovasi ke aspek multimodal dengan menggabungkan analisis data teks dan data klinis dalam memberikan respons. Pendekatan multimodal ini berbeda dari studi sebelumnya yang umumnya berfokus pada modalitas teks saja, seperti chatbot untuk psikiater (Chen dkk., 2023) atau aplikasi LLM dalam radiologi (Bhayana, 2024), sehingga diharapkan dapat meningkatkan interaksi pengguna dan relevansi informasi yang disajikan.

BAB 3

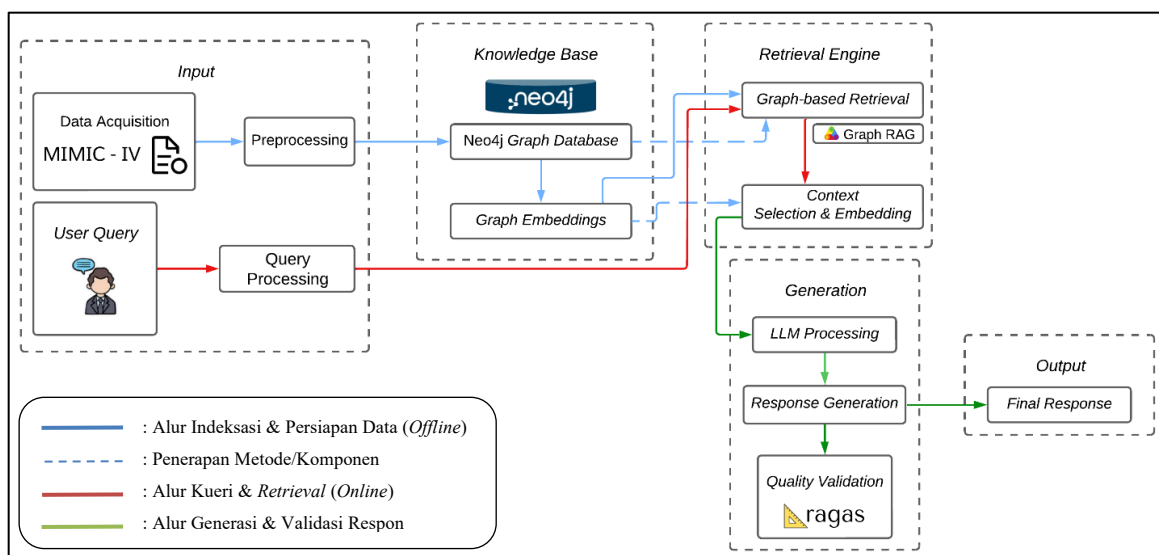
METODOLOGI PENELITIAN

3.1 Data Penelitian

Data yang digunakan ialah MIMIC-IV (*Medical Information Mart for Intensive Care*) versi 3.1, yang dikembangkan oleh *Laboratory for Computational Physiology* (LCP). Dataset bersifat *open-source* yang dapat diakses melalui PhysioNet, melalui beberapa proses kredensial. Dataset berasal dari *Beth Israel Deaconess Medical Center* (BIDMC), sebuah rumah sakit pendidikan yang berafiliasi dengan *Harvard Medical School* di Amerika Serikat, mencakup informasi pasien rawat inap selama periode 2008 - 2019 dengan total lebih dari **382.000 pasien** yang merepresentasikan **523.740 catatan rawat inap**.

3.2 Arsitektur Umum

Gambar 3.1 menunjukkan arsitektur umum dari sistem chatbot biomedis yang dikembangkan menggunakan pendekatan RAG dengan fondasi *Knowledge Graph* (*GraphRAG*). Gambar ini menjelaskan alur kerja sistem secara menyeluruh, mulai dari tahap akuisisi dan pra-pemrosesan data, pembangunan Knowledge Graph di Neo4j, hingga proses retrieval informasi oleh agent. Selanjutnya, informasi tersebut digunakan oleh LLM untuk menghasilkan jawaban, yang kemudian divalidasi kualitasnya sebelum disajikan kepada pengguna.



Gambar 3.1 Arsitektur Umum Penelitian

3.2.1 Dataset

Diawali dengan proses akuisisi dataset dimana penulis diharuskan mengikuti *course* mengenai "*Protecting Human Research Participants*" oleh CITI Program (*Collaborative Institutional Training Initiative*), penandatanganan *Data Use Agreement* (DUA), dan pengajuan permintaan akses disertai dengan justifikasi penelitian yang jelas oleh pembimbing penelitian. Kemudian baru akuisisi data dapat dilakukan melalui *Physionet* setelah *approval* DUA. MIMIC-IV terdiri dari dua modul utama: modul *hosp*, modul ICU. Namun pada penelitian ini penulis hanya akan menggunakan modul *hosp* dimana mencakup data dari sistem *Electronic Health Record* (EHR) pasien. Adapun modul tambahan yang penulis gunakan ialah *note* (MIMIC-IV-Note: *Deidentified free-text clinical notes* 2.2)

3.2.2 Data Preprocessing

Tahap *data preprocessing* dalam penelitian ini bertujuan untuk mengubah data mentah MIMIC-IV yang heterogen menjadi format yang bersih, terstruktur, dan siap untuk dibangun menjadi sebuah *knowledge graph*. Proses ini dibagi menjadi dua alur utama berdasarkan jenis datanya.

1. Data Selection & Filtering – Structured Data

Dilakukan *data selection* yang akan dipakai dalam penelitian ini. Dengan meninjau tujuan penelitian, data yang dipilih ialah data yang memiliki keterkaitan dengan informasi dan kunjungan pasien, diagnosis, prosedur, pengobatan, pegujian lab, dan catatan pasien, yang akan digunakan sebagai dasar untuk *knowledge base* chatbot. Berikut merupakan gambaran singkat dari tabel yang digunakan dalam pembuatan *knowledge graph*.

a. Modul *hosp* :

- i. *admission.csv* ; berisi informasi mengenai pasien yang masuk ke rumah sakit, yang rincian kolomnya disajikan pada Tabel 3.1.

Tabel 3.1 Tabel *admission.csv*

subject_id	hadm_id	admittime	disctime	deathtime	admission_type	admit_provider_id	admission_location	discharge_location
10000032	22595853	2180-05-06 22:23:00	2180-05-07 17:15:00		URGENT	P49AFC	TRANSFER FROM HOSPITAL	HOME
		insurance	language	marital_status	race	edregtime	edouttime	hospital_expire_flag
10000032	22841357	Medicaid	English	WIDOWED	WHITE	2180-05-06 19:17:00	2180-05-06 23:30:00	0
10000032	25742920	Medicaid	English	WIDOWED	WHITE	2180-06-26 15:54:00	2180-06-26 21:31:00	0

- ii. *d_icd_diagnoses* ; berfungsi untuk mendefinisikan kode *International Classification of Diseases* (ICD) versi 9 dan 10, yang pemetaan kode dengan deskripsi diagnosisnya dapat dilihat pada Tabel 3.2.

Tabel 3.2 Tabel *d_icd_diagnoses*

icd_code	icd_version	long_title
0010	9	Cholera due to vibrio cholerae
0011	9	Cholera due to vibrio cholerae el tor

- iii. *d_icd_procedures* ; berisi kode *Klasifikasi Penyakit Internasional* (ICD) untuk setiap prosedur medis, dengan struktur dan contoh datanya terlampir pada Tabel 3.3.

Tabel 3.3 Tabel *d_icd_procedures.csv*

icd_code	icd_version	long_title
0001	9	Therapeutic ultrasound of vessels of head and neck
0002	9	Therapeutic ultrasound of heart

- iv. *d_labitems* ; mendefinisikan setiap item pengukuran laboratorium dalam database MIMIC, di mana rincian untuk setiap itemid beserta label dan kategorinya ditampilkan dalam Tabel 3.4.

Tabel 3.4 Tabel *d_labitems.csv*

itemid	label	fluid	category
50801	Alveolar-arterial Gradient	Blood	Blood Gas
50802	Base Excess	Blood	Blood Gas

- v. *diagnoses_icd* ; berisi catatan semua diagnosis yang ditagihkan kepada pasien, di mana setiap barisnya merepresentasikan satu diagnosis spesifik yang diberikan, seperti terlihat pada Tabel 3.5.

Tabel 3.5 Tabel *diagnoses_icd.csv*

subject_id	hadm_id	seq_num	icd_code	icd_version
10000032	22595853	1	5723	9
10000032	22595853	2	78959	9

- vi. *labevents* ; menyimpan semua hasil pengukuran laboratorium yang dilakukan untuk pasien, dengan contoh hasil pengukurannya untuk beberapa item terlampir di Tabel 3.6.

Tabel 3.6 Tabel *labevents.csv*

labevent_id	subject_id	hadm_id	specimen_id	itemid	order_provider_id	charttime	storetime	value	valuenum	valueuom	ref_range_lower	ref_range_upper	flag	priority	comments
1	10000032		2704548	50931	P69FQC	2180-03-23 11:51:00	2180-03-23 15:56:00	—	95	mg/dL	70	100		ROUTINE	IF FASTING, 70-100 NORMAL, >125 PROVISIONAL DIABETES.
2	10000032		36092842	51071	P69FQC	2180-03-23 11:51:00	2180-03-23 16:00:00	NEG						ROUTINE	

- vii. *patients* ; berisi informasi demografis yang konsisten selama masa hidup pasien, yang rincian data intinya untuk setiap individu dicontohkan pada Tabel 3.7.

Tabel 3.7 Tabel *patients.csv*

subject_id	gender	anchor_age	anchor_year	anchor_year_group	dod
10000032	F	52	2180	2014 - 2016	2180-09-09
10000048	F	23	2126	2008 - 2010	

- viii. *prescriptions* ; memberikan informasi tentang obat yang diresepkan, di mana detail peresepan seperti dosis dan rute pemberiannya dicatat, seperti terilustrasi pada Tabel 3.8.

Tabel 3.8 Tabel *prescriptions.csv*

subject_id	hadm_id	pharmacy_id	poe_id	poe_seq	order_provider_id	starttime	stoptime	drug_type	drug	formulary_drug_cd	gsn	ndc
10000032	22595853	12775705	10000032-55	55	P85UQ1	2180-05-08 08:00:00	2180-05-07 22:00:00	MAIN	Furosemide	FURO40	008209	51079007320
10000032	22595853											
10000032	22595853	prod_strength	form_rx	dose_val_rx	dose_unit_rx	form_val_disp	form_unit_disp	doses_per_24_hrs	route			
10000032	22595853	40mg Tablet		40	mg	1	TAB	1	PO/NG			
		2.5mL Vial		1	NEB	1	VIAL	4	IH			

- ix. *procedures_id* ; berisi catatan semua prosedur yang ditagihkan kepada pasien selama masa rawat inap di rumah sakit dengan menggunakan ontologi ICD-9 dan ICD-10, sebagaimana dicontohkan pada Tabel 3.9.

Tabel 3.9 Tabel *procedures_id*

subject_id	hadm_id	seq_num	chartdate	icd_code	icd_version
10000032	22595853	1	2180-05-07	5491	9
10000032	22841357	1	2180-06-27	5491	9

- b. Modul *note* :

- i. *discharge* ; berisi ringkasan pemulangan dalam bentuk narasi panjang yang menjelaskan alasan pasien masuk, perjalanan penyakit, serta instruksi pemulangan yang relevan. Contoh dari narasi pemulangan ini dapat dilihat pada Tabel 3.10.

Tabel 3.10 Tabel *discharge.csv*

note_id	subject_id	hadm_id	note_type	note_seq	charttime	storetime	text
10000032-DS-21	10000032	22595853	DS	21	2180-05-07 00:00:00	2180-05-09 15:26:00	Name: __ Unit No: __ Admission Date: __ Discharge Date: __ Date of Birth: __ Sex: F Service: MEDICINE Allergies: No Known Allergies / Adverse Drug Reactions Attending: __ Chief Complaint: Worsening ABD distention and pain Major Surgical or Invasive Procedure: Paracentesis History of Present Illness: __ HCV cirrhosis c/b ascites, hiv on ART, h/o IVDU, COPD, bioplar, PTSD, presented from OSH ED with worsening abd distention over past week. Pt reports self-discontinuing lasix and spironolactone __ weeks ago, because she feels like "they don't do anything" and that she "doesn't want to put more chemicals in her." She does not follow Na-restricted diets. In the past week, she notes that she has been having worsening abd distention and discomfort. She denies __ edema, or SOB, or orthopnea. She denies f/c/n/v, d/c, dysuria. She had food poisoning a week ago from eating stale cake (n/v 20 min after food ingestion), which resolved the same day. She denies other recent illness or sick contacts. She notes that she has been noticing gum bleeding while brushing her teeth in recent weeks. she denies easy bruising, melena, BRBPR, hemetesis, hemoptysis, or hematuria. Because of her abd pain, she went to OSH ED and was transferred to __ for further care. Per ED report, pt has brief period of confusion - she did not recall the ultrasound or bloodwork at osh. She denies recent drug use or alcohol use. She denies feeling confused, but reports that she is forgetful at times. In the ED, initial vitals were 98.4 70 105/63 16 97%RA Labs notable for ALT/AST/AP __ __ __ Tbili1.6, WBC 5K, platelet 77, INR 1.6 Past Medical History: 1. HCV Cirrhosis 2. No history of abnormal Pap smears. 3. She had calcification in her breast, which was removed previously and per patient not, it was benign. 4. For HIV disease, she is being followed by Dr. __ Dr. __. 5. COPD 6. Past history of smoking. 7. She also had a skin lesion, which was biopsied and showed skin cancer per patient report and is scheduled for a complete removal of the skin lesion in __ of this year. 8. She also had another lesion in her forehead with purple discoloration. It was biopsied to exclude the possibility of __'s sarcoma, the results is pending. 9. A 15 mm hypochoic lesion on her ultrasound on __ and is being monitored by an MRI. 10. History of dysplasia of anus in __ 11. Bipolar affective disorder, currently manic, mild, and PTSD. 12. History of cocaine and heroin use. Social History: __ Family History: She a total of five siblings, but she is not talking to most of them. She only has one brother that she is in touch with and lives in __. She is not aware of any known GI or liver disease in her family. Her last alcohol consumption was one drink two months ago. No regular alcohol consumption. Last drug use __ years ago. She quit smoking a couple of years ago. Physical Exam: VS: 98.1 107/61 78 16 97%RA General: in NAD HEENT: CTAB, anicteric sclera, OP clear Neck: supple, no LAD CV: RRR, S1S2, no m/r/g Lungs: CTAB, prolonged expiratory phase, no w/r/r Abdomen: distended, mild diffuse tenderness, +flank dullness, cannot percuss liver/spleen edge __ distention GU: no Foley Ext: wwp, no c/e/e. + clubbing Neuro: AAO3, converse normally, able to recall 3 times after 5 minutes, CN II-XII intact Discharge: PHYSICAL EXAMINATION: VS: 98 105/70 95 General: in NAD HEENT: anicteric sclera, OP clear Neck: supple, no LAD CV: RRR, S1S2, no m/r/g Lungs: CTAB, prolonged expiratory phase, no w/r/r Abdomen: distended but improved, TTP in RUQ, GU: no Foley Ext: wwp, no c/e/e. + clubbing Neuro: AAO3, CN II-XII intact Pertinent Results: __ 10:25PM GLUCOSE-109* UREA N-25* CREAT-0.3* SODIUM-138 POTASSIUM-3.4 CHLORIDE-105 TOTAL CO2-27 ANION GAP-9__ 10:25PM estGFR-Using this __ 10:25PM ALTS(GPT)-100* AST(SGOT)-114* ALK PHOS-114* TOT BILU-1.6* __ 10:25PM UPASE-77* __ 10:25PM ALBUMIN-3.3* __ 10:25PM WBC-5.0* RBC-4.29 HGB-14.3 HCT-42.6 MCV-99* MCH-33.3* MCHC-33.5 RDW-15.7* __ 10:25PM

Sehingga total tabel yang digunakan ialah **10 dari 26 tabel** yang ada pada kedua modul. Selanjutnya, proses untuk data terstruktur ini ialah dengan *filtering* pada tabel *prescriptions.csv*. Langkah ini bertujuan untuk melakukan **deduplikasi entitas obat** guna memastikan integritas dan efisiensi *knowledge graph*. Proses ini sangat penting untuk mencegah redundansi, di mana satu jenis obat yang sama direpresentasikan oleh jutaan *node* berbeda.

Implementasinya dilakukan dengan mengumpulkan seluruh data nama obat (drug) dari tabel, kemudian menyaringnya untuk mendapatkan himpunan nama obat yang unik. Proses *filtering* ini secara signifikan mereduksi jumlah entitas

yang perlu dibuat, seperti yang dirangkum pada Tabel 3.11.

Tabel 3.11 Hasil Reduksi Entitas Obat

Deskripsi	Jumlah
Jumlah total baris pada <code>prescriptions.csv</code>	20.292.610
Jumlah nama obat unik (setelah deduplikasi)	10.926
Jumlah node :Medication yang dibuat	10.926

Dengan demikian, dari lebih dari 20 juta catatan peresepan, sistem hanya perlu membuat dan mengelola 10.926 node :Medication yang unik. Dengan demikian, fokus pembersihan data terstruktur diprioritaskan pada proses deduplikasi entitas master untuk menjaga integritas dan efisiensi graf. Untuk tabel-tabel transaksional lain seperti *diagnoses_icd.csv* dan *labevents.csv*, tidak dilakukan pembersihan data secara ekstensif. Hal ini dikarenakan data tersebut sebagian besar berisi kode dan ID standar dari *dataset* MIMIC-IV yang sudah terkurasi, di mana tugas utamanya adalah sebagai penghubung antar *node*. Oleh karena itu, data tersebut dinilai sudah cukup layak untuk dimuat secara langsung guna merepresentasikan hubungan data apa adanya.

2. Segmentasi Data Teksual (*Text Chunking*)

Data preprocessing tidak terstruktur—khususnya pada catatan medis dari *discharge.csv*—merupakan langkah krusial dalam arsitektur *Retrieval-Augmented Generation* (RAG) yang dibangun. Catatan klinis seringkali berisi teks naratif yang sangat panjang, yang dapat melebihi batas token masukan dari *Large Language Model* (LLM) dan tidak efisien untuk proses pencarian semantik. Oleh karena itu, diperlukan sebuah metode untuk memecah teks tersebut menjadi segmen-segmen yang lebih kecil dan mudah dikelola tanpa kehilangan informasi kontekstual yang penting.

Untuk mengatasi tantangan ini, penelitian ini mengimplementasikan teknik **segmentasi teks (*text chunking*)**. Teknik ini diimplementasikan menggunakan pustaka `RecursiveCharacterTextSplitter()` dari *LangChain*. Secara spesifik, setiap teks dari *node* :Note dipecah menjadi fragmen-fragmen

(*chunks*) dengan konfigurasi berikut:

- **chunk_size**: 1000 karakter. Ukuran ini dipilih sebagai titik optimal untuk memastikan setiap fragmen cukup padat secara informasional namun tetap berada dalam batas pemrosesan yang efisien.
- **chunk_overlap**: 100 karakter. Tumpang tindih antar fragmen diterapkan untuk menjaga kesinambungan konteks dan memastikan tidak ada informasi penting yang terpotong di perbatasan antar-fragmen.

Hasil dari proses segmentasi ini bukanlah teks yang dimodifikasi, melainkan sebuah daftar fragmen teks yang terstruktur untuk setiap catatan medis. Fragmen-fragmen inilah yang kemudian siap untuk diproses pada tahap selanjutnya, yaitu pembuatan representasi vektor (*embedding*), yang akan menjadi fondasi untuk mekanisme pencarian semantik.

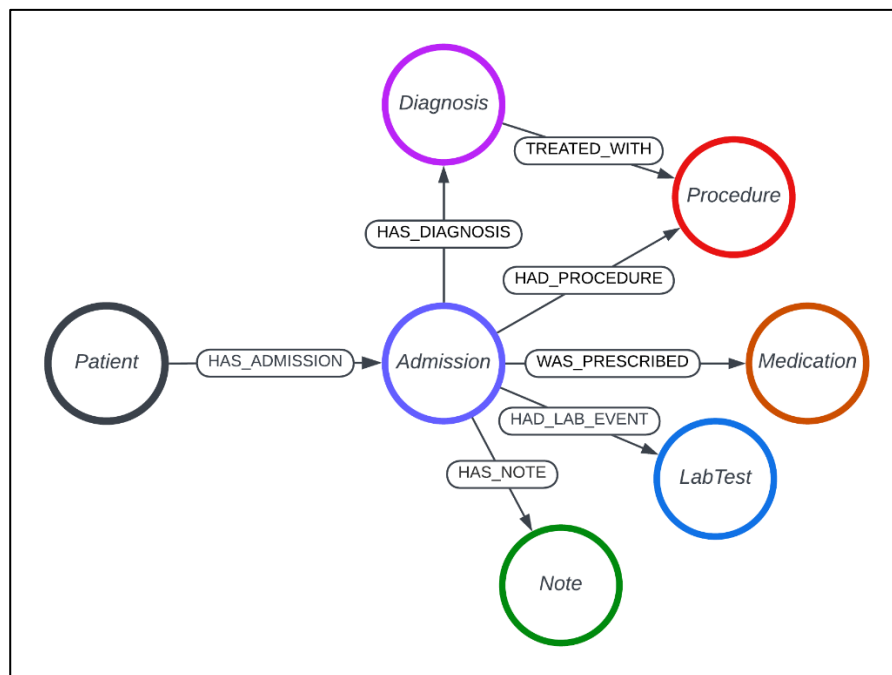
Tabel 3.12 Ilustrasi Segmentasi Catatan Medis Pasien

Contoh Isi Kolom Text	Sample Segmentasi	Label Segmentasi (<i>Chunk</i>)
“ ... <i>She also had a skin lesion, which was biopsied and showed skin cancer per patient report and is scheduled for a complete removal of the skin lesion in of this year. 8. She also had another lesion in her forehead with purple discoloration. It was biopsied to exclude the possibility of 's sarcoma, the results is pending. 9. A 15 mm hypoechoic lesion on her ultrasound on and is being monitored by an MRI.</i> ... “	<i>“She also had a skin lesion, which was biopsied and showed skin cancer per patient report and is scheduled for a complete removal of the skin lesion in ____ of this year. 8. She also had”</i>	<i>Chunk 1</i>
	<i>“She also had another lesion in her forehead with purple discoloration. It was biopsied to exclude the possibility of ____'s sarcoma,”</i>	<i>Chunk 2</i>
	<i>“the possibility of ____'s sarcoma, the results is</i>	<i>Chunk 3</i>

	pending. 9. A 15 mm hypoechoic lesion on her ultrasound on ____ and is being monitored by an MRI.”	
--	---	--

3.2.3 Pembangunan Knowledge Graph

Tahap awal dalam pembangunan *knowledge graph* adalah inialisasi *node-node*. Tujuan dari tahap ini adalah untuk mempopulasi *database* graf dengan entitas-entitas biomedis fundamental yang akan berfungsi sebagai "kamus" data. Proses ini memastikan bahwa setiap entitas unik, seperti informasi pasien atau prosedur, direpresentasikan oleh satu *node* tunggal untuk menjaga konsistensi dan integritas data.. Adapun arsitektur basis data yang dibuat dapat dilihat pada Gambar 3.2 di bawah.



Gambar 3.2 Arsitektur *node & relationship* basis data

1. Inialisasi *Node* Master

Proses inialisasi dilakukan untuk beberapa entitas utama sebagai berikut:

- **Node** :Diagnosis : Data dari file kamus *d_icd_diagnoses.csv* digunakan sebagai sumber. Setiap baris dalam file ini merepresentasikan

satu kode diagnosis yang unik (berdasarkan *icd_code* dan *icd_version*). Menggunakan perintah *Cypher* **MERGE**, setiap kode diagnosis tersebut dibuat menjadi sebuah *node :Diagnosis* yang unik. Properti tambahan seperti *long_title* juga disematkan ke dalam *node* untuk menyimpan deskripsi lengkap dari diagnosis.

- **Node :Procedure** : Dengan pola yang sama, *node :Procedure* dibuat dari file *d_icd_procedures.csv*. Setiap kode prosedur unik menjadi satu *node* di dalam graf.
- **Node :LabTest** : Data dari *d_labitems.csv* digunakan untuk membuat *node :LabTest*. Setiap *node* diidentifikasi secara unik oleh *itemid* dan menyimpan properti seperti label (nama tes) dan *fluid* (jenis spesimen).
- **Node :Medication** : Berbeda dari entitas lain yang memiliki file kamus sendiri, *node* obat diekstrak langsung dari data transaksional *prescriptions.csv*. Untuk memastikan tidak ada duplikasi, dilakukan proses **deduplikasi** terlebih dahulu seperti yang dijelaskan sebelumnya. Kemudian, perintah **MERGE** digunakan untuk membuat satu *node :Medication* untuk setiap nama obat yang unik tersebut.

Setelah langkah ini selesai, *knowledge graph* telah terisi dengan entitas-entitas biomedis fundamental yang akan menjadi jangkar untuk menghubungkan data transaksional pasien pada tahap selanjutnya. Berikut *pseudo-code* untuk inisialisasi *node* master.

Fondasi dari sistem ini adalah sebuah *knowledge graph* yang dibangun secara sistematis dari data mentah MIMIC-IV. Proses konstruksi ini diimplementasikan melalui tiga langkah, yakni :

- Langkah 1: Inisialisasi Entitas Master - Tahap awal berfokus pada pembentukan kamus data utama di dalam Neo4j. Implementasi ini menggunakan kueri **MERGE** untuk setiap entitas, seperti *:Diagnosis* dan *:Procedure*, yang berfungsi untuk membuat *node* jika belum ada dan memastikan tidak terjadi duplikasi data. Properti deskriptif seperti *long_title* juga ditambahkan ke setiap *node* menggunakan klausa **SET** untuk memperkaya data.
- Langkah 2: Pembuatan Hubungan Transaksional - Setelah entitas *master* tersedia, maka eksekusi selanjutnya ialah untuk membangun struktur inti dari

graf menggunakan sampel **100.000 data admisi**. Pada tahap ini, node `:Admission` ditetapkan sebagai titik pusat (*central hub*) yang menghubungkan data `:Patient` dengan semua kejadian klinisnya. Secara sistematis membentuk *relationship* yang menghubungkan `:Admission` dengan node-*node* master yang relevan, seperti `:HAS_DIAGNOSIS` dan `:HAD_PROCEDURE`.

- Langkah 3: Integrasi Data Tekstual - Tahap konstruksi terakhir adalah mengintegrasikan data naratif dari catatan klinis. Membuat node `:Note` untuk setiap catatan medis dari **100.000 admisi**, kemudian membentuk hubungan `:HAS_NOTE` yang menautkan setiap catatan tersebut ke *node* `:Admission` yang sesuai. berikut ditampilkan *pseudocode* untuk proses inisiasi *node* master pada Gambar 3.3.

```

ON event: Jalankan pipeline Master Data
  DEFINE konstanta:
    - NEO4J_URI, NEO4J_AUTH
    - DATA_PATH

  INITIALIZE driver Neo4j
  IF koneksi ke Neo4j FAILED
  STOP eksekusi
  ENDIFF

  READ "d_icd_diagnoses.csv" ke dalam diagnosa_df
  FOR setiap baris dalam diagnosa_df:
    EXECUTE Cypher "MERGE (d:Diagnosa {kode_icd: baris.kode_icd,
...}) SET d.long_judul = baris.long_judul"
  ENDF OR

  READ "d_icd_procedures.csv" into procedures_df
  FOR setiap baris dalam procedures_df:
    EXECUTE Cypher "MERGE (p:Prosedur {kode_icd: row.kode_icd, ...})
SET p.long_judul = row.long_judul"
  ENDFOR

  READ "d_labitems.csv" into lab_items_df
  FOR setiap baris dalam lab_items_df:
    EXECUTE Cypher "MERGE (l:LabTest {itemid: row.itemid}) SET
l.label = row.label, l.cairan = row.cairan"
  ENDFOR

  READ "resep.csv" into resep_df
  GET obat_unik dari resep_df
  FOR setiap nama_obat dalam obat_unik:
    EXECUTE Cypher "MERGE (m:Obat {nama: nama_obat})"
  AKHIRI

  TUTUP koneksi driver Neo4j
END

```

Gambar 3.3 *Pseudocode* proses Inisialisasi Node Master (Kamus Data)

2. Pembuatan Hubungan Transaksional

Setelah *node-node master* berhasil diinisialisasi, tahap selanjutnya adalah membangun struktur inti dari *knowledge graph* dengan menciptakan hubungan antar entitas berdasarkan data transaksional pasien. Proses ini bertujuan untuk menghubungkan setiap pasien dengan catatan kunjungan rawat inap mereka, serta semua kejadian klinis yang relevan selama masa perawatan tersebut. Proses ini dilakukan melalui beberapa langkah berurutan:

a) **Pembuatan Node Pasien dan Kunjungan** : Langkah pertama adalah

membuat *node* untuk entitas inti transaksional. Data dari *patients.csv* dan *admissions.csv* dibaca untuk membuat *node :Patient* yang unik untuk setiap pasien (diidentifikasi oleh *subject_id*) dan *node :Admission* untuk setiap catatan rawat inap (diidentifikasi oleh *hadm_id*).

- b) **Pembentukan Hubungan Inti (*Patient* → *Admission*)** : Setelah *node* pasien dan kunjungan dibuat, hubungan pertama yang dibentuk adalah $(p:Patient) - [:HAS_ADMISSION] \rightarrow (a:Admission)$.

Hubungan ini sangat fundamental karena menjadi "tulang punggung" dari graf, yang mengaitkan setiap pasien ke riwayat kunjungan mereka di rumah sakit.

- c) **Menghubungkan Data Klinis ke Kunjungan**: Langkah berikutnya adalah memperkaya setiap *node :Admission* dengan menghubungkannya ke *node-node master* yang relevan berdasarkan kejadian selama masa rawat inap tersebut. Proses ini dilakukan dengan pola MATCH (temukan *node* sumber dan tujuan) dan MERGE (buat hubungan jika belum ada). Pola ini diterapkan untuk beberapa jenis data:

- **Hubungan Diagnosis** : Data *diagnoses_icd.csv* digunakan untuk menghubungkan *node :Admission* dengan *node :Diagnosis* yang sesuai, membentuk relasi $[:HAS_DIAGNOSIS]$.
- **Hubungan Prosedur** : Dengan pola serupa, *procedures_icd.csv* digunakan untuk membentuk relasi $[:HAD_PROCEDURE]$.
- **Hubungan Peresepan Obat** : Data *prescriptions.csv* digunakan untuk membentuk relasi $[:PRESCRIBED_MEDICATION]$.
- **Hubungan Hasil Lab**: Terakhir, *labevents.csv* digunakan untuk membentuk relasi $[:HAD_LAB_EVENT]$. Pada hubungan ini, properti seperti *valuenum* (nilai hasil tes) dan *valueuom* (satuan) juga ditambahkan langsung pada relasi untuk menyimpan informasi spesifik dari kejadian tersebut.

Setelah semua langkah ini selesai, graf kini memiliki struktur yang kaya, di mana setiap kunjungan pasien terhubung secara kontekstual dengan diagnosis, prosedur, obat, dan hasil tes lab yang terjadi selama periode tersebut. Berikut ditampilkan kembali *pseudocode* untuk proses pembuatan hubungan

transaksional pada Gambar 3.4.

```

ON event: Jalankan pipeline Data Transaksional
  DEFINE constants:
    - NEO4J_URI, NEO4J_AUTH, DATA_PATH

  INITIALIZE Neo4j driver
  IF connection to Neo4j FAILED
    STOP execution
  ENDIF

  // Langkah 1: Buat Node :Patient dan :Admission
  READ "patients.csv" into patients_df
  FOR each row in patients_df:
    EXECUTE Cypher "MERGE (:Patient {subject_id: row.subject_id})"
  ENDFOR

  READ "admissions.csv" into admissions_df
  FOR each row in admissions_df:
    EXECUTE Cypher "MERGE (:Admission {hadm_id: row.hadm_id})"
  ENDFOR

  // Langkah 2: Buat Hubungan Antar Node
  // Hubungan (Patient)-[:HAS_ADMISSION]->(Admission)
  FOR each row in admissions_df:
    MATCH (p:Patient {subject_id: row.subject_id})
    MATCH (a:Admission {hadm_id: row.hadm_id})
    EXECUTE Cypher "MERGE (p)-[:HAS_ADMISSION]->(a)"
  ENDFOR

  // Hubungan (Admission)-[:HAS_DIAGNOSIS]->(Diagnosis)
  READ "diagnoses_icd.csv" into diagnoses_df
  FOR each row in diagnoses_df:
    MATCH (a:Admission {hadm_id: row.hadm_id})
    MATCH (d:Diagnosis {icd_code: row.icd_code})
    EXECUTE Cypher "MERGE (a)-[:HAS_DIAGNOSIS]->(d)"
  ENDFOR

  // Hubungan (Admission)-[:HAD_PROCEDURE]->(Procedure)
  READ "procedures_icd.csv" into procedures_df
  FOR each row in procedures_df:
    MATCH (a:Admission {hadm_id: row.hadm_id})
    MATCH (pr:Procedure {icd_code: row.icd_code})
    EXECUTE Cypher "MERGE (a)-[:HAD_PROCEDURE]->(pr)"
  ENDFOR

  // Hubungan (Admission)-[:PRESCRIBED_MEDICATION]->(Medication)
  READ "prescriptions.csv" into prescriptions_df
  FOR each row in prescriptions_df:
    MATCH (a:Admission {hadm_id: row.hadm_id})
    MATCH (m:Medication {name: row.drug})
    EXECUTE Cypher "MERGE (a)-[:PRESCRIBED_MEDICATION]->(m)"
  ENDFOR

  // Hubungan (Admission)-[:HAD_LAB_EVENT]->(LabTest) dengan Properti
  READ "labevents.csv" into labevents_df
  FOR each row in labevents_df:
    MATCH (a:Admission {hadm_id: row.hadm_id})
    MATCH (l:LabTest {itemid: row.itemid})
    EXECUTE Cypher "MERGE (a)-[:HAD_LAB_EVENT {valuenum:
row.valuenum, ...}]->(l)"
  ENDFOR

  CLOSE Neo4j driver connection
ENDEND

```

Gambar 3.4 Pseudocode proses Pembuatan Node Transaksional & Hubungannya

3. Integrasi Data Tidak Terstruktur (Catatan Klinis)

Langkah terakhir adalah memperkaya *knowledge graph* dengan data

tekstual dari catatan klinis, yang merupakan sumber informasi kontekstual yang sangat kaya.

- Pembuatan Node Catatan: Setiap catatan dari `noteevents.csv` dijadikan sebuah node `:Note`. Properti yang disimpan pada node ini mencakup `text`, `category`, dan `description` dari catatan tersebut.
- Menghubungkan Catatan ke Kunjungan: Setiap node `:Note` kemudian dihubungkan ke node `:Admission` yang relevan melalui hubungan `[:HAS_NOTE]`. Hal ini memastikan bahwa setiap catatan klinis terikat secara kontekstual dengan episode rawat inap spesifik dari seorang pasien.

Berikut ditampilkan juga *pseudocode* untuk integrasi data tidak terstruktur (catatan klinis) pada Gambar 3.5.

```

ON event: Jalankan pipeline Data Catatan (Note)
  DEFINE constants:
    - NEO4J_URI, NEO4J_AUTH
    - DATA_PATH
    - SAMPLE_SIZE

  INITIALIZE Neo4j driver
  IF connection to Neo4j FAILED
    STOP execution
  ENDIF

  READ "noteevents.csv" into notes_df

  FOR each row in notes_df:
    // Langkah 3a: Buat Node untuk setiap Catatan
    EXECUTE Cypher "CREATE (n:Note {note_id:
row.note_id, text: row.text, ...})"

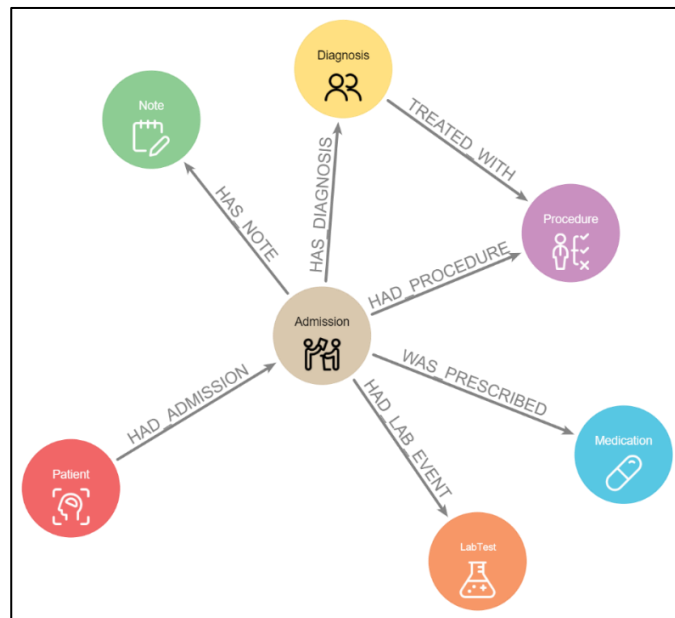
    // Langkah 3b: Hubungkan Node Catatan ke Kunjungan
    MATCH (n:Note {note_id: row.note_id})
    MATCH (a:Admission {hadm_id: row.hadm_id})
    EXECUTE Cypher "MERGE (a)-[:HAS_NOTE]->(n)"
  ENDFOR

  CLOSE Neo4j driver connection
END

```

Gambar 3.5 *Pseudocode* proses Integrasi Data Tidak Terstruktur (Catatan Klinis)

Hasil akhir dari ketiga langkah implementasi ini adalah sebuah *knowledge graph* yang koheren dan siap untuk tahap selanjutnya. Statistik kuantitatif dari graf yang berhasil dibangun disajikan pada Tabel 3.13. *Node* dan *relationship* yang telah diimpor ke dalam basis data juga dapat divisualisasikan menggunakan Neo4j Bloom. Visualisasi basis data ini dapat dilihat pada Gambar 3.6 *Node* berwarna merah muda menunjukkan *subject_id* pasien, *node* berwarna krem menunjukkan id kunjungan, *node* hijau menunjukkan id catatan klinis, *node* berwarna oranye menunjukkan uji lab yang dilakukan, *node* biru muda menunjukkan obat/resep yang diberikan, *node* ungu menunjukkan prosedur yang diterima/dilakukan oleh pasien, dan *node* kuning menunjukkan diagnosa yang diterima pasien.



Gambar 3.6 Visualisasi Sampel Data Setelah di Muat ke Neo4j

Tabel 3.13 Statistik Akhir *Knowledge Graph*

Entitas	Keterangan	Jumlah
<i>Node</i>	:Patient	41384
	:Admission	100000
	:Diagnosis	111749
	:Procedure	86423
	:Medication	5211
	:LabTest	1650
	:Note	60766
Total <i>Node</i>		407183
	:HAS_ADMISSION	100000

<i>Relationship</i>	:HAS_DIAGNOSIS	101843
	:HAD_PROCEDURE	95065
	:WAS_PRESCRIBED	54723
	:HAD_LAB_EVENT	4498
	:HAS_NOTE	60766
	:TREATED_WITH	129501
Total Relationship		546396

3.2.4 Pengayaan Semantik dan Indeksasi Vektor

Setelah struktur relasional dari *knowledge graph* berhasil dibangun, tahap implementasi selanjutnya adalah pengayaan semantik (*semantic enrichment*). Tujuan dari tahap ini adalah untuk mengubah data tekstual tidak terstruktur—yang tersimpan dalam node `:Note`—menjadi format vektor yang dapat dipahami dan diukur kemiripannya oleh mesin. Proses ini merupakan prasyarat fundamental untuk mengaktifkan kemampuan pencarian semantik pada alur kerja RAG.

Implementasi pengayaan ini dijalankan melalui beberapa langkah komputasi yang terdefinisi dengan jelas:

1. **Segmentasi Teks (*Text Chunking*)**: menargetkan properti *text* dari setiap *node* `:Note`. Mengingat catatan klinis dapat sangat panjang, setiap teks dipecah menjadi segmen-segmen yang lebih kecil dan tumpang-tindih (*overlapping chunks*) menggunakan kelas *RecursiveCharacterTextSplitter* dari *framework* LangChain.
2. **Generasi *Embedding***: Setiap segmen teks yang telah dibuat kemudian ditransformasikan menjadi representasi vektor numerik berdimensi tinggi (*embedding*). Proses ini memanfaatkan model *text-embedding-004* dari Google, yang dipanggil melalui kelas *GoogleGenerativeAIEmbeddings* di LangChain. Hasil dari langkah ini adalah serangkaian vektor yang secara matematis merepresentasikan makna semantik dari setiap segmen teks.
3. **Penyimpanan *Embedding* pada Graf**: Vektor-vektor yang telah digenerasi kemudian disimpan kembali ke dalam *knowledge graph*. Skrip mengeksekusi kueri *Cypher MATCH* dan *SET* untuk menambahkan properti baru bernama *embedding* pada setiap *node* `:Note` yang sesuai. Properti ini berisi daftar vektor yang merepresentasikan keseluruhan isi dari catatan klinis tersebut.

4. **Pembuatan Indeks Vektor:** Skrip menjalankan kueri *Cypher* **CREATE VECTOR INDEX**, yang sintaks dan konfigurasinya ditunjukkan pada Gambar 3.7, untuk membuat sebuah indeks khusus bernama *note_embeddings* pada properti *embedding* dari semua *node* dengan *label* :Note.

```
CREATE VECTOR INDEX note_embeddings IF NOT EXISTS
FOR (n:Note) ON (n.embedding)
OPTIONS { indexConfig: {
  `vector.dimensions`: 768,
  `vector.similarity_function`: 'cosine'
}}
```

Gambar 3.7 Konfigurasi Indeks Vektor

Melalui *indexConfig*, kita dapat mengontrol aspek teknis dari indeks, seperti dimensi vektor dan metrik kemiripan yang akan digunakan. *`vector.dimensions: 768`* : Parameter ini mendefinisikan dimensi atau panjang dari setiap vektor yang akan diindeks. Angka 768 bukanlah nilai yang arbitrer; nilai ini harus sama persis dengan dimensi *output* dari model embedding yang digunakan, yaitu *text-embedding-004* dari Google. Kesesuaian dimensi ini bersifat wajib untuk memastikan bahwa basis data dapat menyimpan dan memproses vektor secara benar. Ketidakcocokan dimensi akan menyebabkan kegagalan dalam proses pengindeksan atau perhitungan kemiripan. *`vector.similarity\_function: 'cosine'`* : Parameter ini menentukan algoritma matematis yang digunakan untuk mengukur kemiripan antara dua vektor.

Dalam pemrosesan bahasa alami, *cosine similarity* adalah metrik yang sangat umum dan efektif. Alasannya adalah karena metrik ini fokus pada "arah" dari vektor (yang merepresentasikan konteks semantik) dan mengabaikan "besaran" atau magnitudonya. Hal ini membuatnya andal dalam membandingkan kemiripan makna antara dua teks, bahkan jika panjang kedua teks tersebut sangat berbeda. Dalam konteks RAG, ini memungkinkan sistem untuk menemukan segmen catatan medis yang relevan secara semantik dengan pertanyaan pengguna, terlepas dari perbedaan panjang kalimat.

3.2.5 Agent Graph Retrieval-Augmented Generation (GraphRAG)

Setelah *knowledge graph* berhasil dibangun dan dipopulasi dengan data, tahap selanjutnya adalah mengimplementasikan logika inti dari sistem *chatbot*. Sebelum *agent* dapat berfungsi sebagai orkestrator, dua rantai logika (*chains*) yang sangat terspesialisasi diimplementasikan sebagai komponen. Setiap komponen dirancang dengan alur kerja dan sumber daya spesifik untuk menangani jenis-jenis pertanyaan yang berbeda secara fundamental.

- a. Komponen Kueri Graf (*graphrag_cypher*) : Komponen ini dibangun di atas kelas *GraphCypherQAChain* dari *framework* LangChain. *Tool* ini bertanggung jawab untuk menjawab pertanyaan faktual, kuantitatif, dan spesifik yang jawabannya dapat diekstrak secara langsung dari struktur relasional *knowledge graph*. Saat *graph_cypher_chain* dieksekusi, serangkaian proses komputasi terjadi secara berurutan:
 - *Input Pengguna*: Menerima pertanyaan dalam bahasa alami, misalnya, "Apa obat paling umum yang digunakan untuk penyakit gagal jantung?".
 - *Generasi Kueri Cypher*: Pertanyaan tersebut, beserta skema *knowledge graph* yang secara otomatis diambil oleh Neo4jGraph, dikirim ke sebuah instance LLM (*gemini-2.5-pro-preview-06-05*) yang bertugas khusus untuk penerjemahan.
 - *Analisis Prompt Template Cypher*: Prompt yang digunakan (*cypher_generation_prompt*) menginstruksikan LLM untuk bertindak sebagai seorang "pakar Pengembang Neo4j". *Prompt* ini memberikan aturan ketat: hanya gunakan *node labels* dan *relationship types* yang ada di dalam skema, jangan mengarang, dan konversikan pertanyaan pengguna menjadi kueri Cypher yang valid.
 - *Validasi dan Eksekusi*: Kueri *Cypher* yang dihasilkan dari *GraphCypherQAChain* pada contoh pertanyaan di atas ialah berikut pada Gambar 3.8.

```

> Entering new GraphCypherQAChain chain...
Generated Cypher:
MATCH (d:Diagnosis)-[:HAS_DIAGNOSIS]-(a:Admission)-[:WAS_PRESCRIBED]->(m:Medication)
WHERE d.Long_title CONTAINS 'heart failure'
RETURN m.name, count(m) AS frequency
ORDER BY frequency DESC
LIMIT 1
Full Context:
[{'m.name': 'Sodium Chloride 0.9% Flush', 'frequency': 625}]

```

Gambar 3.8 Contoh *log* proses *tool*

Maka hasil kueri akan divalidasi dahulu secara sintaksis

(*validate_cypher=True*) sebelum dieksekusi pada basis data Neo4j.

- Sintesis Jawaban: Data mentah (*Full Context*) yang dikembalikan oleh Neo4j kemudian diteruskan ke instance LLM kedua (*gemini-2.5-pro-preview-06-05*) bersama dengan pertanyaan asli.
- Generasi Jawaban Akhir: Instance LLM kedua ini, dipandu oleh *qa_generation_prompt*, bertugas merangkai data mentah tersebut menjadi sebuah jawaban naratif yang koheren dan mudah dipahami oleh pengguna.

- b. Komponen Analisis Teks Naratif (*review_chain*) : Komponen ini dibangun menggunakan kelas *RetrievalQA* dari *LangChain*. *Tool* ini dirancang untuk menjawab pertanyaan kualitatif, interpretatif, atau berbasis ringkasan yang jawabannya terkubur di dalam data tekstual tidak terstruktur (catatan medis pada node *:Note*). Contohnya termasuk "Bagaimana ringkasan perawatan pasien X?" atau "Apakah ada riwayat merokok yang disebutkan dalam catatan?". *Retriever* dibuat dari *instance* *Neo4jVector*, yang telah terhubung ke indeks vektor *note_embeddings*.

```
retriever = vector_store.as_retriever()
```

Ketika *retriever* ini dieksekusi dengan sebuah pertanyaan, berikut adalah alur algoritma yang terjadi:

- Vektorisasi *Input*: Pertanyaan pengguna (misalnya, "ringkasan kondisi pasien dengan sirosis hati") diterima sebagai input string. String ini kemudian diubah menjadi sebuah vektor numerik menggunakan model *text-embedding-004* pada *GoogleGenerativeAIEmbeddings*.
- *Similarity Search* : Vektor pertanyaan ini dikirim ke basis data Neo4j. Neo4j kemudian memanfaatkan indeks vektor *note_embeddings* untuk melakukan

operasi pencarian kemiripan.

- Eksekusi Algoritma Kemiripan: Sesuai konfigurasi indeks yang kita bahas sebelumnya (*vector.similarity_function: 'cosine'*), Neo4j menghitung skor *cosine similarity* antara vektor pertanyaan dengan semua vektor chunk teks yang ada di dalam indeks.
- *Chunk Retrieval* : *retriever* akan mengambil *chunk* teks dari node :*Note* yang memiliki skor kemiripan tertinggi.

Pemanggilan `vector_store.as_retriever()` tanpa argumen tambahan akan menggunakan parameter *default* LangChain yang paling umum, terdapat 2 argumen yang disesuaikan yakni ;

- `search_type` : dengan *value* yang digunakan ialah “similarity”.
- `search_kwargs` : dengan *parameter value* `k = 2` (mengambil 2 *chunks* teratas untuk catatan medis relevan).

Setelah kedua rantai logika terspesialisasi (*graphrag_cypher* dan *review_chain*) siap, tahap implementasi selanjutnya adalah mengintegrasikan keduanya ke dalam sebuah agent tunggal yang koheren. *Agent* ini berfungsi sebagai orkestrator atau "otak" sistem yang secara dinamis memilih dan mengeksekusi tool yang paling tepat untuk menjawab pertanyaan pengguna. Proses perakitan ini melibatkan dua langkah utama: mendefinisikan *tools* dengan deskripsi yang jelas dan merancang *prompt* utama yang akan menjadi panduan bagi LLM dalam mengambil keputusan.

a. *Tool 1: note_reviewer*

- Fungsi yang dieksekusi : `review_chain.invoke` - fungsi ini memanggil rantai logika *RetrievalQA* yang telah kita bahas sebelumnya, yang melakukan pencarian semantik pada indeks vektor catatan medis.
- **Deskripsi tool** : "*WAJIB digunakan untuk semua pertanyaan yang membutuhkan pemahaman mendalam dari catatan naratif atau teks bebas. Ini adalah satu-satunya cara untuk mengetahui ringkasan perawatan, kondisi pasien dari waktu ke waktu, atau cerita di balik admisi. Sangat ideal untuk pertanyaan seperti: 'Bagaimana ringkasan perawatan untuk pasien pneumonia?', 'Jelaskan kondisi pasien dengan penyakit atau tindakan ...', atau 'Apa saja observasi pasca-operasi*

untuk prosedur bypass jantung?' "

b. *Tool 2: graphrag_cypher*

- Fungsi yang dieksekusi : `graphrag_cypher.invoke` - Fungsi ini memanggil rantai *GraphCypherQAChain* yang menerjemahkan bahasa alami ke kueri Cypher, mengeksekusinya, dan mensintesis jawaban.
- **Deskripsi tool** : *"WAJIB digunakan untuk semua pertanyaan objektif dan kuantitatif tentang data rumah sakit. Ini adalah satu-satunya cara untuk mendapatkan data pasien, statistik, dan informasi medis yang akurat dan terverifikasi dari database graf. Gunakan untuk pertanyaan yang mengandung kata kunci seperti 'berapa banyak', 'jumlah', 'daftar', 'rata-rata', '... apa', atau statistik. Contoh: 'Berapa jumlah total pasien yang didiagnosis dengan diabetes?', 'Sebutkan 5 prosedur yang paling sering dilakukan', 'Siapa dokter yang menangani pasien X?', 'obat apa yang diresepkan untuk penyakit ...', 'tindakan medis apa saja yang paling umum dilakukan untuk ...?'"*

Perilaku dan proses pengambilan keputusan ini diatur secara eksplisit melalui sebuah *prompt template* utama yang bernama `hospital_agent_prompt`. *Prompt* ini berfungsi sebagai piagam instruksi (*instruction charter*) yang memandu logika internal LLM.

a. *Struktur Prompt Template*

Prompt utama ini dibangun menggunakan kelas *ChatPromptTemplate* dari *LangChain*, yang menstrukturkan interaksi menjadi serangkaian pesan yang telah ditentukan. Implementasi menggunakan tiga jenis pesan utama:

1. **Pesan Sistem (*system*)**: Berisi instruksi inti, persona, aturan, dan batasan yang harus selalu dipatuhi oleh *agent*. Ini adalah "konstitusi" bagi *agent*.
2. **Pesan Pengguna (*human*)**: Sebuah *placeholder* (`{input}`) yang akan diisi dengan pertanyaan aktual dari pengguna saat *runtime*.
3. **Pesan *Placeholder* (*placeholder*)**: Sebuah *placeholder* khusus (`{agent_scratchpad}`) yang berfungsi sebagai "memori kerja" atau "catatan internal" bagi *agent*.

b. Struktur Pesan Sistem (*Prompt Engineering*)

Pesan sistem adalah komponen yang paling kritikal karena di sinilah logika *routing* dan perilaku *agent* didefinisikan secara eksplisit.

- **Definisi Persona:** *Prompt* diawali dengan menetapkan identitas dan tujuan dari *agent*. *"Anda adalah asisten AI biomedis yang canggih. Tujuan utama Anda adalah memberikan jawaban yang akurat dan dapat dipercaya berdasarkan data terverifikasi yang ada di dalam basis data internal."*
- **Aturan Pemilihan Tool (Logika Routing):** Bagian selanjutnya memberikan instruksi yang sangat tegas mengenai logika pemilihan *tool*. Ini adalah "algoritma" pengambilan keputusan yang harus diikuti oleh LLM.
- **Aturan untuk note_reviewer :** *"Jika pertanyaan bersifat kualitatif (misalnya 'bagaimana kondisi pasien', 'ringkasan perawatan', 'apa yang terjadi selama admisi X'), KAMU HARUS menggunakan tool note_reviewer."*
- **Aturan untuk graphrag_cypher :** *"Jika pertanyaan bersifat kuantitatif atau faktual (misalnya 'berapa banyak', 'daftar', 'siapa dokter', 'apa obat untuk X'), KAMU HARUS menggunakan tool graphrag_cypher."*
- **Aturan Penanganan Jawaban:** *Prompt* juga mendefinisikan cara *agent* harus bersikap saat menjawab, termasuk menangani pertanyaan di luar konteks. *"Jika sebuah pertanyaan bersifat umum dan tidak ada di dalam tools (mis. 'apa itu demam?'), Anda boleh menjawab dari pengetahuan umum, tetapi berikan disclaimer: 'Informasi ini bersifat umum dan tidak berasal dari database chatbot.'" "Selalu gunakan Bahasa Indonesia yang sopan dan mudah dimengerti."*

Dengan merancang *prompt* yang detail dan sarat dengan aturan-aturan eksplisit ini, ketergantungan pada kemampuan penalaran "bebas" dari LLM dapat diminimalkan. Sebagai gantinya, LLM dipaksa untuk bertindak sebagai *router* yang patuh pada logika yang telah ditetapkan, sehingga meningkatkan konsistensi dan keandalan sistem.

c. *Assembly Final* dengan *AgentExecutor*

Proses ini melibatkan dua fungsi utama dari *framework* LangChain.

1. Pembuatan *Agent* Logis (*create_tool_calling_agent*)

Pertama, sebuah *agent* logis dibuat menggunakan fungsi *create_tool_calling_agent*. Fungsi ini berperan sebagai "perekat" yang mengikat tiga elemen inti menjadi satu:

- **LLM** : yang akan melakukan penalaran.
- **Prompt**: *hospital_agent_prompt* yang berisi semua instruksi dan aturan.
- **Tools**: Daftar tools yang telah kita definisikan sebelumnya.

Hasil dari fungsi ini adalah sebuah objek *agent* yang mengandung logika inti untuk penalaran dan pemilihan *tool*, namun belum dapat dieksekusi secara langsung.

2. Inisialisasi Eksekutor (*AgentExecutor*)

Untuk menjalankan *agent* logis tersebut, sebuah *runtime environment* bernama *AgentExecutor* diinisialisasi. *AgentExecutor* adalah komponen yang bertanggung jawab atas seluruh siklus hidup eksekusi sebuah permintaan, mulai dari menerima input pengguna hingga mengembalikan jawaban akhir.

```
hospital_rag_agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    return_intermediate_steps=True,
    verbose=True,
    handle_parsing_errors=True,
)
```

Berikut adalah konfigurasi parameter yang digunakan:

- ***agent=agent*** : Menentukan objek *agent* logis yang telah dibuat pada langkah sebelumnya untuk dieksekusi.
- ***tools=tools*** : Menyediakan daftar tools yang dapat diakses dan dieksekusi oleh eksekutor.
- ***return_intermediate_steps=True*** : Dengan mengaturnya ke *True*, eksekutor tidak hanya akan mengembalikan jawaban akhir, tetapi juga semua langkah perantara yang diambil oleh *agent* (misalnya, *tool* apa yang dipanggil, apa inputnya, dan apa *output* dari *tool* tersebut). Informasi inilah yang kemudian digunakan untuk menampilkan *log* pada fitur "*How was this generated*" di antarmuka pengguna.

- ***verbose=True*** : Parameter ini digunakan untuk keperluan pengembangan dan *debugging*. Saat diaktifkan, *AgentExecutor* akan mencetak seluruh alur pemikiran dan eksekusi *agent* ke konsol secara *real-time*.
- ***handle_parsing_errors=True*** : Jika LLM sesekali menghasilkan *output* yang tidak sesuai format dan tidak dapat diurai oleh *parser* LangChain, parameter ini menginstruksikan eksekutor untuk menangani kesalahan tersebut secara elegan alih-alih menyebabkan seluruh sistem berhenti bekerja.

Dengan terciptanya *hospital_rag_agent_executor*, keseluruhan *backend* sistem kini siap untuk menerima permintaan, melakukan penalaran multi-langkah yang kompleks, dan menghasilkan jawaban yang berbasis data.

3.2.6 Evaluasi Kinerja *GraphRAG*

Untuk mengukur efektivitas dan keandalan dari sistem chatbot yang dibangun, diperlukan sebuah kerangka evaluasi yang sistematis. Penelitian ini mengadopsi kerangka kerja RAGAS yang menyediakan serangkaian metrik standar untuk menilai kualitas dari alur *Retrieval-Augmented Generation* (RAG), dengan evaluasi ini penulis tidak bertujuan untuk menguji komponen secara terpisah, melainkan untuk menilai kinerja **sistem *agent* secara holistik dan *end-to-end***. Evaluasi akan difokuskan pada empat metrik utama dari RAGAS yang secara holistik dapat mengukur kualitas sistem dari dua perspektif: kinerja komponen *retrieval* dan kinerja komponen *generation*.

- Metrik untuk kualitas generasi (*Generation Quality*) :
 - i. ***Answer Correctness*** : Metrik ini adalah tolak ukur utama untuk akurasi faktual, yang membandingkan jawaban yang dihasilkan sistem dengan jawaban ideal (*ground truth*) yang telah disiapkan. Metrik ini menilai sejauh mana jawaban sistem sejalan dengan kebenaran faktual. Secara matematis, ini dihitung menggunakan F1-Score yang menyeimbangkan presisi dan *recall* dari pernyataan yang dibuat.

$$\text{Answer Correctness} = \frac{TP}{TP + 0.5 \times (FP + FN)}$$

Di mana:

- TP (*True Positive*): Pernyataan dalam jawaban yang benar dan sesuai dengan *ground truth*.

- FP (*False Positive*): Pernyataan dalam jawaban yang salah atau tidak ada dalam ground truth.
 - FN (*False Negative*): Pernyataan dalam ground truth yang tidak disebutkan dalam jawaban.
- ii. ***Faithfulness*** : Metrik ini menilai apakah jawaban yang dihasilkan oleh sistem sepenuhnya didukung secara faktual oleh konteks yang diambil (*retrieved_context*). Ini adalah metrik krusial untuk mengukur tingkat halusinasi dari model. Secara matematis, *faithfulness* dihitung dengan rumus:

$$Faithfulness = \frac{|V|}{|S|}$$

Di mana:

- |V| adalah jumlah pernyataan dalam jawaban yang terverifikasi benar berdasarkan konteks.
 - |S| adalah total jumlah pernyataan dalam jawaban.
- iii. ***Answer_relevancy*** : Metrik ini mengukur seberapa relevan jawaban yang dihasilkan terhadap pertanyaan asli. Metrik ini dihitung dengan mengukur kemiripan kosinus (*cosine similarity*) antara *embedding* pertanyaan asli dengan *embedding* dari serangkaian pertanyaan hipotetis yang digenerasi dari jawaban. Rumus yang digunakan adalah:

$$Answer\ Relevancy = \frac{1}{n} \sum_{i=1}^n sim(q, q_i)$$

Keterangan :

- q adalah representasi vektor (*embedding*) dari pertanyaan asli yang diajukan oleh pengguna.
 - q_i adalah representasi vektor dari pertanyaan hipotetis ke-i yang digenerasi oleh LLM berdasarkan jawaban yang telah dibuat.
 - n adalah jumlah total pertanyaan hipotetis yang digenerasi.
 - sim adalah fungsi kemiripan kosinus (*cosine similarity*) yang mengukur kesamaan sudut antara dua vektor. Skor yang mendekati 1 menandakan kemiripan yang sangat tinggi.
- Metrik Kualitas Retrieval (*Retrieval Quality*)
 - i. ***context_precision*** : mengevaluasi relevansi dari konteks yang diambil,

mengukur seberapa banyak informasi yang "berguna" dibandingkan dengan informasi yang "tidak perlu".

$$\text{Context Precision} = \frac{\sum_{k=1}^K (\text{Precision}@k \times v_k)}{\text{Jumlah total item yang relevan dalam hasil teratas } K}$$

Dengan :

$$\text{Precision}@k = \frac{\text{true positives}@k}{(\text{true positives}@k + \text{false positives}@k)}$$

Di mana, K adalah jumlah total *chunk* dalam *retrieved_contexts* dan $v_k \in \{0,1\}$ adalah indikator relevansi pada peringkat k .

- ii. **context_recall** : menilai apakah *retriever* berhasil mengambil semua informasi yang diperlukan dari *knowledge base* untuk menjawab pertanyaan secara komprehensif.. Skor yang rendah dapat mengindikasikan bahwa beberapa fakta atau potongan teks penting mungkin terlewatkan selama proses pencarian.

$$\text{Context Precision} = \frac{|\text{Jumlah kalimat ground truth yang didukung konteks}|}{|\text{Total kalimat dalam ground truth}|}$$

A. Kerangka Kerja dan Metrik Evaluasi

Pemilihan RAGAS didasarkan pada kemampuannya dalam menyediakan evaluasi multi-aspek yang holistik terhadap sistem RAG, mencakup kualitas komponen retrieval dan generation secara terpisah, serta mampu meminimalkan subjektivitas melalui penilaian berbasis Large Language Model (LLM). Performa sistem diukur menggunakan lima metrik kuantitatif utama yang telah tervalidasi di atas.

B. Desain Eksperimen Berbasis Evaluasi Komponen

Mengingat karakteristik arsitektur sistem yang bersifat hibrid, desain eksperimen tidak menggunakan pendekatan perbandingan dengan baseline eksternal, melainkan mengadopsi metodologi evaluasi berbasis tugas komponen (component-based task evaluation). Berdasarkan atribut *expected_tool* yang telah

didefinisikan dalam dataset evaluasi, dirancang dua skenario pengujian utama:

- i. Skenario 1: Evaluasi Performa Tool Faktual-Graf (*graphrag_cypher*)
Skenario ini menggunakan subset pertanyaan dari dataset yang secara spesifik dirancang untuk dijawab melalui penelusuran graf terstruktur. Tujuan evaluasi adalah mengukur efektivitas rantai GraphCypherQACHain dalam melakukan translasi pertanyaan natural language ke query Cypher dan mengambil data faktual dari knowledge graph.
- ii. Skenario 2: Evaluasi Performa Tool Naratif-Vektor (*note_reviewer*)
Skenario ini menggunakan subset pertanyaan yang memerlukan pemahaman dan sintesis informasi dari teks naratif tidak terstruktur. Tujuan evaluasi adalah mengukur efektivitas rantai *RetrievalQA* dalam menemukan konteks relevan melalui pencarian semantik berbasis *vector similarity*.

Selain pengukuran metrik RAGAS pada kedua skenario, akurasi routing dari agent juga dianalisis untuk mengukur kemampuannya dalam melakukan delegasi pertanyaan ke tool yang tepat berdasarkan karakteristik query.

C. Dataset Evaluasi

Evaluasi sistem dilakukan menggunakan dataset kurasi manual yang tersimpan dalam file `eval_dataset.csv`. Dataset ini terdiri dari 35 pertanyaan yang telah dirancang untuk mencakup spektrum kasus penggunaan yang komprehensif dalam domain biomedis berdasarkan data MIMIC IV v3.1. Setiap entri dalam dataset memiliki dua atribut esensial:

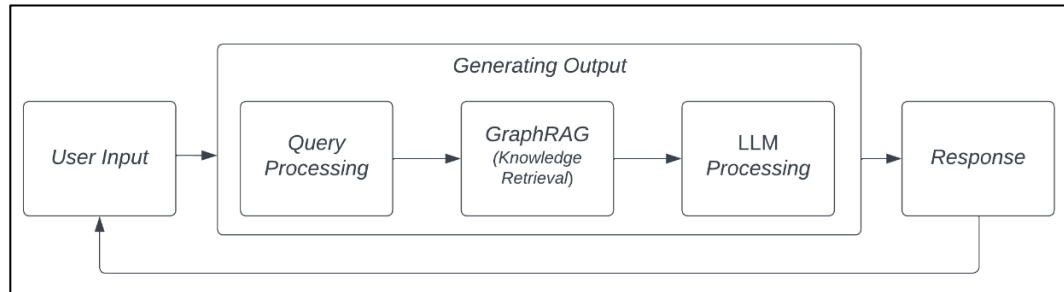
- *question*: Pertanyaan yang akan diproses oleh sistem agent
- *ground_truth*: Jawaban ideal atau referensi faktual yang diharapkan

3.3 Perancangan Sistem

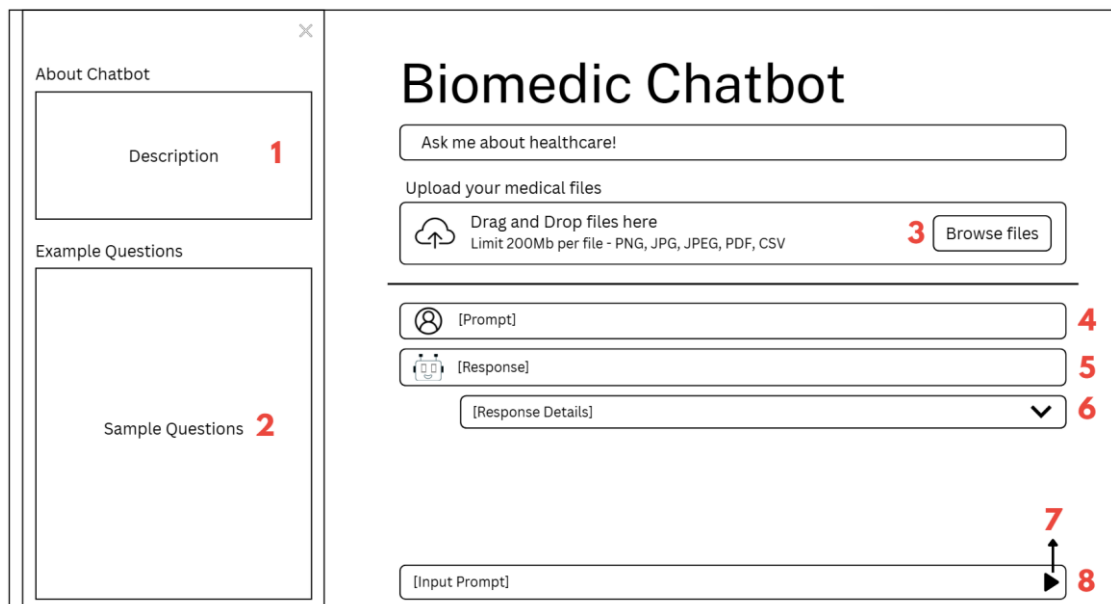
3.3.1 Rancangan Antarmuka Sistem

Pada bagian ini akan dijelaskan mengenai desain antarmuka sistem untuk *chatbot* biomedis yang dikembangkan. Perancangan ini bertujuan untuk menyediakan media interaksi yang intuitif dan mudah digunakan bagi pengguna untuk berinteraksi dengan sistem *Retrieval-Augmented Generation* (RAG) yang telah dibangun. Sistem ini dirancang sebagai aplikasi berbasis web dengan tampilan tunggal (*single-page application*) untuk memaksimalkan kemudahan

akses dan penggunaan. Adapun alur sistem yang dibangun dapat dilihat pada gambar 3.9 dan Rancangan antarmuka sistem secara keseluruhan dapat dilihat pada Gambar 3.10 berikut.



Gambar 3.9 Flowchart Sistem



Gambar 3.10 Desain Antarmuka

Keterangan :

Label 1 : Deskripsi singkat mengenai chatbot

Label 2 : Daftar contoh pertanyaan yang bisa

Label 3 : Merupakan tempat upload file berekstensi png, jpg, jpeg, pdf, dan csv pada website

Label 4 : *Textbox* menampilkan *prompt* yang telah dimasukkan oleh *user*

Label 5 : *Textbox* menampilkan *response* yang di-generate

Label 6 : *box* untuk menampilkan detail mengenai *retrieval* dalam *response*

Label 7 : Tombol untuk mengirim *input*

Label 8 : *Textbox* yang menampung input teks pengguna yang akan dideteksi

BAB 4

IMPLEMENTASI DAN PENGUJIAN SISTEM

Pada tahap implementasi, rancangan sistem yang telah diuraikan pada Bab 3 diwujudkan menjadi sebuah aplikasi fungsional. Proses ini melibatkan penyiapan lingkungan pengembangan yang mencakup spesifikasi perangkat keras dan lunak, serta penerapan arsitektur sistem menjadi antarmuka pengguna yang interaktif. Implementasi ini bertujuan untuk memastikan bahwa setiap komponen, mulai dari *knowledge graph* hingga alur kerja GraphRAG, dapat berjalan secara terintegrasi dan sesuai dengan tujuan penelitian.

4.1 Spesifikasi Perangkat Keras dan Perangkat Lunak

Pelaksanaan penelitian ini didukung oleh serangkaian perangkat keras (hardware) dan perangkat lunak (software), berikut adalah spesifikasi perangkat keras yang dipakai untuk mengembangkan sistem dalam penelitian ini:

1. ASUS Vivobook S14 OLED K3402ZA
2. Processor 12th Gen Intel(R) Core(TM) i5-12500H 2.50 GHz
3. RAM 16 GB

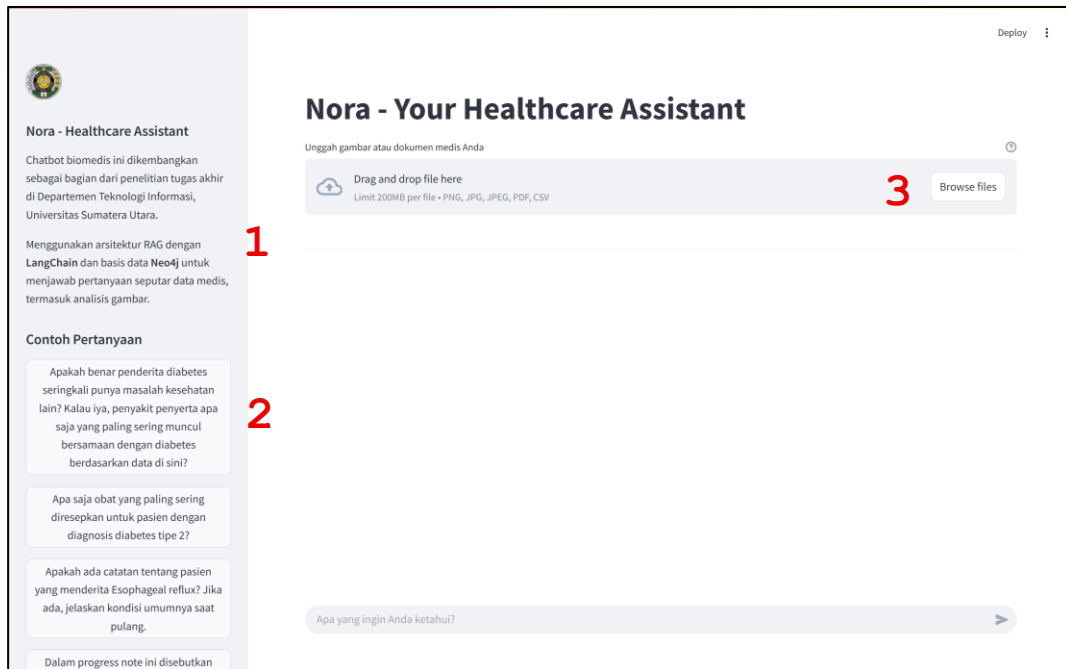
Adapun spesifikasi perangkat lunak yang dipakai untuk mengembangkan sistem dalam penelitian ini:

1. OS: Windows 11 Home Single Language 24H2 (64-bit)
2. Bahasa Pemrograman *Python* versi 3.12.2
3. IDE: *Microsoft Visual Studio Code*
4. *Graph Database Neo4j Desktop* 1.6.2
5. LangChain v0.3
6. Model LLM dan *Embedding* :
 - Model LLM : *Google Gemini 2.5 Pro*
 - Model *Embedding* : *Google text-embedding-004*

4.2 Implementasi Rancangan Antarmuka

Antarmuka sistem atau user interface (UI) direalisasikan sebagai sebuah aplikasi web interaktif menggunakan framework Streamlit. Implementasi ini secara fungsional dan visual mengacu pada rancangan yang telah dipaparkan pada Gambar 3.18.

Aplikasi ini dirancang sebagai halaman tunggal (*single-page application*) yang memuat seluruh komponen interaksi dalam satu tampilan, seperti yang ditunjukkan pada Gambar 4.1.



Gambar 4.1 Tampilan Antarmuka Chatbot

Berikut adalah penjelasan implementasi dari setiap komponen utama pada antarmuka sistem:

- Komponen *About* (Label 1) ; berisi deskripsi singkat mengenai arsitektur chatbot, yang menjelaskan bahwa sistem ini "berinteraksi dengan agent LangChain dan menggunakan metode retrieval-augment generation (RAG) pada data terstruktur dan tidak terstruktur".
- Komponen Contoh Pertanyaan (Label 2) ; diimplementasikan dengan menampilkan daftar pertanyaan konkret untuk memandu pengguna, seperti "Ada berapa total pasien di database?", "Berapa jumlah pasien yang mengalami rawat inap lebih dari 7 hari?", dan serupa lainnya.
- Area Unggah File Multimodal (Label 3): Sesuai dengan rancangan, fitur untuk menangani input multimodal diimplementasikan sebagai area *drag-and-drop* atau *browse file*. Komponen ini memungkinkan pengguna untuk mengunggah berkas medis dalam format .PNG, .JPG, .JPEG, .PDF, dan .CSV untuk dianalisis oleh sistem.

Gambar 4.2 Tampilan Antarmuka Chatbot (Lanjutan)

- d. Area Interaksi Utama (Label 4 & 5): Bagian ini merupakan jendela percakapan yang menampilkan dialog antara pengguna dan chatbot. Pertanyaan dari pengguna (Label 4) dan respons yang dihasilkan oleh chatbot (Label 5) ditampilkan secara berurutan, menciptakan alur percakapan yang logis dan mudah diikuti.
- e. *Detail Response* / Transparansi Agent (Label 6): Fitur "*How was this generated*" (Label 6) diimplementasikan sebagai menu dropdown yang dapat diperluas dan menjadi komponen kunci untuk transparansi sistem. Seperti yang ditunjukkan pada Gambar 4.1 dan 4.2, ketika menu ini dibuka, sistem akan menampilkan log eksekusi dari LangChain agent. Log ini secara detail menunjukkan tool yang dipilih oleh agent (misalnya, tool='graph') dan tool_input yang merupakan kueri yang dieksekusi berdasarkan pertanyaan pengguna. Fungsionalitas ini sangat krusial untuk validasi dan pemahaman proses pengambilan keputusan internal sistem RAG.
- f. Area Input Pengguna (Label 7 & 8): Di bagian bawah halaman, terdapat textbox input (Label 8) dengan placeholder "apa yang ingin anda ketahui" sebagai tempat pengguna mengetikkan pertanyaan mereka. Di sebelahnya, terdapat tombol kirim (Label 7) untuk mengeksekusi prompt dan memulai alur kerja chatbot.

4.3 Hasil dan Analisis Kinerja

Evaluasi terhadap sistem chatbot medis dilakukan dengan menggunakan framework RAGAS terhadap golden dataset yang telah dikembangkan. Proses evaluasi menghasilkan serangkaian skor kuantitatif untuk setiap metrik pada setiap kategori pertanyaan yang telah ditetapkan. Sebelum masuk ke analisis mendalam per kategori pertanyaan, berikut adalah ringkasan performa sistem secara keseluruhan di semua metrik evaluasi untuk setiap pertanyaan pada *golden dataset*.

Tabel 4.1 Ringkasan Skor Rata-Rata Evaluasi RAGAS per Kategori

<i>Question</i>	<i>Faithfulness</i>	<i>Answer Relevancy</i>	<i>Context Precision</i>	<i>Context Recall</i>	<i>Answer Correctness</i>	<i>Category</i>
Berapa jumlah total pasien dalam database?	1	0.8558	1	1	0.9916	<i>Graphrag</i>
Berapa banyak admisi yang masuk melalui Gawat Darurat (URGENT)?	1	0.8754	1	1	0.9892	<i>Graphrag</i>
Bagaimana distribusi gender dari semua pasien?	1	0.8487	1	1	0.9826	<i>Graphrag</i>
Sebutkan 5 diagnosis yang paling sering muncul di semua admisi.	1	0.877	1	1	0.8717	<i>Graphrag</i>
Tampilkan 10 obat yang paling sering diresepkan.	1	0.7512	1	0	0.9029	<i>Graphrag</i>
Untuk pasien berusia di atas 65 tahun, sebutkan 5 prosedur yang	1	0.9231	1	1	0.9432	<i>Graphrag</i>

<i>Question</i>	<i>Faithfulness</i>	<i>Answer Relevancy</i>	<i>Context Precision</i>	<i>Context Recall</i>	<i>Answer Correctness</i>	<i>Category</i>
paling umum.						
Berapa rata-rata usia pasien saat mereka masuk rumah sakit?	1	0.7844	1	1	0.8269	<i>Graphrag</i>
Berapa hari rata-rata lama seorang pasien dirawat?	1	0.7682	1	1	0.7314	<i>Graphrag</i>
Berapa hari terlama dan tercepat seorang pasien pernah dirawat?	1	0.7122	1	1	0.7892	<i>Graphrag</i>
Berapa banyak pasien dengan diagnosis 'Congestive Heart Failure' yang juga memiliki diagnosis 'Diabetes Mellitus'?	1	0.821	1	1	0.9631	<i>Graphrag</i>
Berapa jumlah pasien dengan diagnosis Pneumonia yang diresepkan Aspirin selama masa perawatannya?	1	0.8799	1	1	0.7249	<i>Graphrag</i>
Tampilkan persentase admisi untuk setiap tipe admisi (admission type).	1	0.7843	1	1	0.8426	<i>Graphrag</i>
Berapa banyak admisi yang melibatkan lebih	0.3333	0.8178	1	1	0.9794	<i>Graphrag</i>

<i>Question</i>	<i>Faithfulness</i>	<i>Answer Relevancy</i>	<i>Context Precision</i>	<i>Context Recall</i>	<i>Answer Correctness</i>	<i>Category</i>
dari 7 prosedur medis?						
Untuk admisi gawat darurat, berapa rata-rata jumlah obat yang diresepkan?	1	0.8524	1	1	0.6339	<i>Graphrag</i>
Berapa total akumulasi hari rawat inap untuk semua pasien laki-laki (gender 'M')?	1	0.8319	1	1	0.9422	<i>Graphrag</i>
Bagaimana distribusi tipe admisi untuk pasien yang berusia lebih dari 80 tahun?	1	0.7893	1	0.8	0.8572	<i>Graphrag</i>
Secara total, berapa kali obat 'Warfarin' diresepkan di seluruh database?	1	0.693	1	1	0.9635	<i>Graphrag</i>
Berapa banyak pasien yang tidak memiliki catatan prosedur medis sama sekali selama semua admisi mereka?	1	0.8499	1	1	0.9929	<i>Graphrag</i>
Buatkan rangkuman medis untuk admisi pasien dengan admisi 22595853	0.96774 19	0.8106	1	0	0.8745	<i>Graphrag</i>
Dengan menggunakan kasus	0.96296 3	0.7638	1	0.4	0.5883	<i>Note Reviewer</i>

<i>Question</i>	<i>Faithfulness</i>	<i>Answer Relevancy</i>	<i>Context Precision</i>	<i>Context Recall</i>	<i>Answer Correctness</i>	<i>Category</i>
dari admisi 21086876, jelaskan kepada orang awam apa itu 'Sepsis'.						
Bagaimana respons pasien pada admisi 27897940 terhadap pemberian obat 'Atorvastatin'?	1	0.8907	1	0.7	0.8085	<i>Note Reviewer</i>
Buatkan poin-poin utama dari instruksi kepulangan untuk pasien dengan admisi 22841357.	0.9615385	0.6343	0	0.78	0.1849	<i>Note Reviewer</i>
Berdasarkan catatan medis admisi 22841357, mengapa obat 'Spironolactone' dihentikan?	1	0.8437	1	1	0.5592	<i>Note Reviewer</i>
Jelaskan keluhan utama dan riwayat penyakit sekarang dari pasien pada admisi 25742920.	1	0.5682	0	0.33333333	0.7716	<i>Note Reviewer</i>
Buatkan rangkuman mengenai riwayat sosial pasien pada admisi 25742920.	1	0	1	1	0.8311	<i>Note Reviewer</i>
Sebutkan temuan-temuan kunci dari pemeriksaan fisik saat pasien masuk untuk admisi	1	0.8279	1	1	0.6816	<i>Note Reviewer</i>

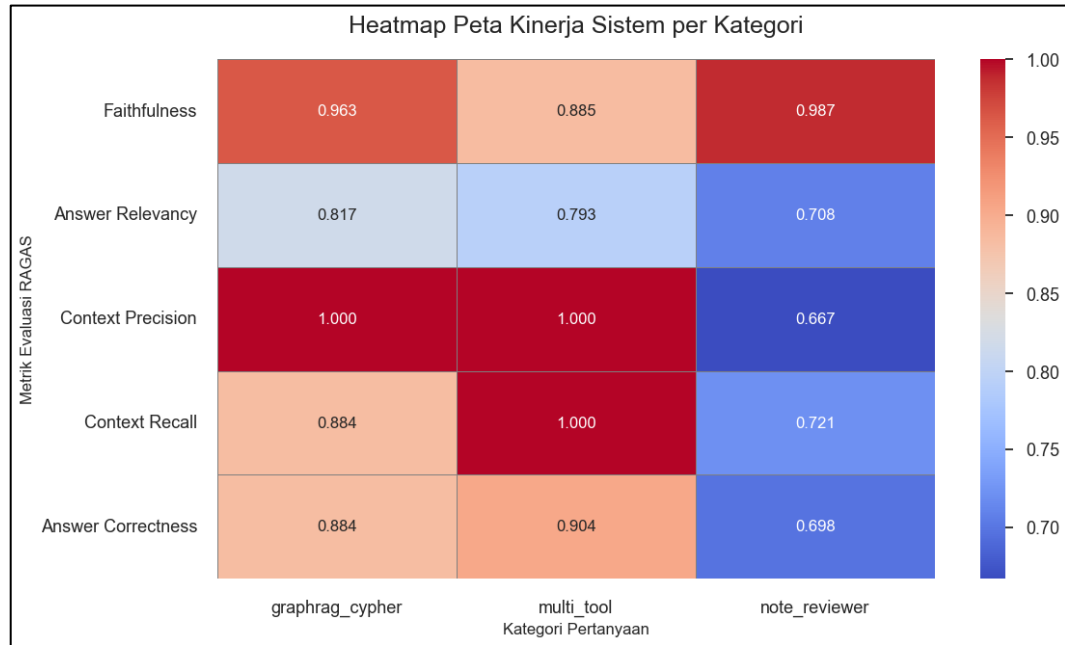
<i>Question</i>	<i>Faithfulness</i>	<i>Answer Relevancy</i>	<i>Context Precision</i>	<i>Context Recall</i>	<i>Answer Correctness</i>	<i>Category</i>
25742920.						
Apa status kode resusitasi (code status) untuk pasien pada admisi 21086876?	1	0.7321	0	0	0.2026	<i>Note Reviewer</i>
Sebutkan alergi obat yang tercatat untuk pasien pada admisi 21086876.	1	0.7432	1	1	0.7132	<i>Note Reviewer</i>
Apa status kode resusitasi pasien pada admisi 25742920 dan jelaskan diskusinya?	1	0.89	0	1	0.8094	<i>Note Reviewer</i>
Sebutkan alergi yang dimiliki oleh pasien pada admisi 22841357.	1	0.729	1	1	0.9844	<i>Note Reviewer</i>
Bagaimana status fungsi ginjal pasien pada admisi 25742920 berdasarkan catatan medis?	1	0.569	0	0	0.8865	<i>Note Reviewer</i>
Pada kasus admisi 27897940, jelaskan peran dan alasan klinis penggunaan obat 'Metoprolol' pada pasien ini.	0.9230769	0.8557	1	0.6	0.6062	<i>Note Reviewer</i>
Analisis riwayat keluarga pasien dari	1	0.8247	1	1	0.8806	<i>Note Reviewer</i>

<i>Question</i>	<i>Faithfulness</i>	<i>Answer Relevancy</i>	<i>Context Precision</i>	<i>Context Recall</i>	<i>Answer Correctness</i>	<i>Category</i>
admisi 27897940 dan jelaskan relevansinya dengan diagnosis utamanya.						
<i>AVERAGE SCORE</i>	0.9709	0.7697	0.8571	0.8175	0.8052	-

Dari Tabel 4.1 di atas, hasil evaluasi sistem secara keseluruhan menunjukkan performa yang cukup kuat dan konsisten. Sistem mencapai skor *faithfulness* sebesar 0.970, yang mengindikasikan bahwa jawaban yang dihasilkan bersifat faktual dan minim halusinasi. *Answer relevancy* berada di angka 0.769, sedangkan *context precision* (0.857) dan *context recall* (0.818) menunjukkan bahwa sistem mampu menemukan dan memanfaatkan konteks yang relevan secara akurat dari basis data. Akurasi jawaban akhir, yang tercermin dari metrik *answer correctness*, juga tergolong baik dengan skor 0.805.

Untuk memperkuat analisis, Gambar 4.3 menyajikan visualisasi *heatmap* kinerja sistem berdasarkan lima metrik utama RAGAS pada tiga kategori pertanyaan: *graphrag_cypher*, *note_reviewer*, dan *multi_tool*. Terlihat bahwa sistem paling unggul pada kategori *graphrag_cypher*, terutama dalam *context precision* (1.00) dan *faithfulness* (0.963), yang menunjukkan efektivitas sistem dalam mengekstrak dan menyajikan informasi dari knowledge graph. Sebaliknya, performa pada kategori *note_reviewer*, yang menangani teks naratif, masih cukup rendah, khususnya pada *context precision* (0.667) dan *answer correctness* (0.698), yang mengindikasikan tantangan dalam memahami teks tidak terstruktur. Sementara itu, kategori *multi_tool* menunjukkan performa yang seimbang dengan skor tinggi pada *context precision* dan *context recall* (1.000), meskipun relevansi jawaban (0.793) masih belum optimal.

Secara umum, hasil ini menegaskan bahwa arsitektur GraphRAG bekerja paling efektif untuk pertanyaan faktual berbasis struktur graf, namun masih memerlukan peningkatan pada pemrosesan konten naratif dalam domain biomedical.



Gambar 4.3 heatmap evaluasi skor metrik

4.3.1 Hasil dan Analisis Kinerja Kuantitatif

Kinerja sistem pada kategori pertanyaan *graphrag_cypher* dirangkum dalam Tabel 4.2 di bawah ini. Tabel ini menyajikan skor rata-rata yang dicapai oleh sistem untuk setiap metrik RAGAS, yang dihitung berdasarkan *golden dataset*.

A. Performa Sistem pada Skenario Faktual-Graf (*graphrag_cypher*)

Tabel 4.2 Skor Rata-Rata Metrik RAGAS untuk Skenario Faktual-Graf

Metrik	Evaluasi Skor	Rata-Rata Interpretasi
<i>Faithfulness</i>	0.985	Tingkat faktualitas jawaban terhadap konteks
<i>Answer Relevancy</i>	0.962	Relevansi jawaban terhadap pertanyaan
<i>Context Precision</i>	1	Presisi informasi dalam konteks yang diambil
<i>Context Recall</i>	0.884	Kelengkapan informasi yang berhasil diambil
<i>Answer Correctness</i>	0.884	Akurasi faktual jawaban akhir

Hasil evaluasi pada Tabel 4.2 mengindikasikan bahwa sistem mampu mencapai performa yang konsisten pada metrik *answer_correctness* dan *faithfulness*. Hal ini mendemonstrasikan efektivitas rantai *GraphCypherQACchain* dalam menghasilkan *query Cypher* yang tepat dan melakukan sintesis jawaban yang akurat berdasarkan data struktural yang berhasil diambil dari *knowledge*

graph. *Context Precision* (0.100) dan *Context Recall* (0.884) menunjukkan bahwa komponen *GraphCypherQACChain* mampu mengonversi *natural language query* menjadi *query Cypher* yang valid untuk ekstraksi data yang presisi. Tingginya skor *Answer Correctness* (0.884) relatif terhadap kategori lain juga memvalidasi efektivitas penggunaan data terstruktur untuk menjawab pertanyaan faktual.

B. Performa Sistem pada Skenario Naratif-Vektor (*note_reviewer*)

Skenario ini mengukur performa sistem pada subset pertanyaan yang memerlukan pemahaman mendalam terhadap konten dari teks medis tidak terstruktur. Evaluasi difokuskan pada kemampuan sistem dalam melakukan pencarian semantik dan mengekstrak informasi relevan dari teks naratif yang kompleks. Tabel 4.3 menyajikan skor rata-rata metrik RAGAS untuk kategori pertanyaan naratif-vektor.

Tabel 4.3 Skor Rata-Rata Metrik RAGAS untuk Skenario Naratif-Vektor

Metrik	Evaluasi Skor	Rata-Rata Interpretasi
<i>Faithfulness</i>	0.987	Tingkat faktualitas jawaban terhadap konteks
<i>Answer Relevancy</i>	0.708	Relevansi jawaban terhadap pertanyaan
<i>Context Precision</i>	0.667	Presisi informasi dalam konteks yang diambil
<i>Context Recall</i>	0.721	Kelengkapan informasi yang berhasil diambil
<i>Answer Correctness</i>	0.698	Akurasi faktual jawaban akhir

Dari tabel 4.4, *Context Precision* (0.667) dan *Context Recall* (0.721) menjadi indikator utama. Meskipun sistem mampu menjaga *Faithfulness* jawaban terhadap konteks yang diambil, kualitas konteks yang kurang presisi tersebut secara langsung menurunkan skor *Answer Correctness* (0.698).

C. Analisis Akurasi *Routing Tool* oleh *Agent*

Komponen kritis dalam arsitektur agent adalah kemampuannya untuk melakukan pemilihan *tool* yang tepat secara otonom berdasarkan karakteristik pertanyaan yang diterima. Akurasi *routing tool* diukur melalui perbandingan antara *tool* yang aktual digunakan oleh agent selama eksekusi dengan nilai *expected_tool* yang telah

ditetapkan dalam dataset evaluasi. Hasil analisis menunjukkan bahwa agent berhasil mencapai **akurasi 100%** dalam memilih *tool* yang tepat untuk seluruh 35 pertanyaan. Hasil ini memberikan validasi yang kuat terhadap efektivitas desain *prompt engineering* dan spesifikasi fungsional *tool* dalam mengarahkan proses *reasoning* agent. Tabel 4.4 menampilkan analisis akurasi pemilihan tool berdasarkan kategori pertanyaan.

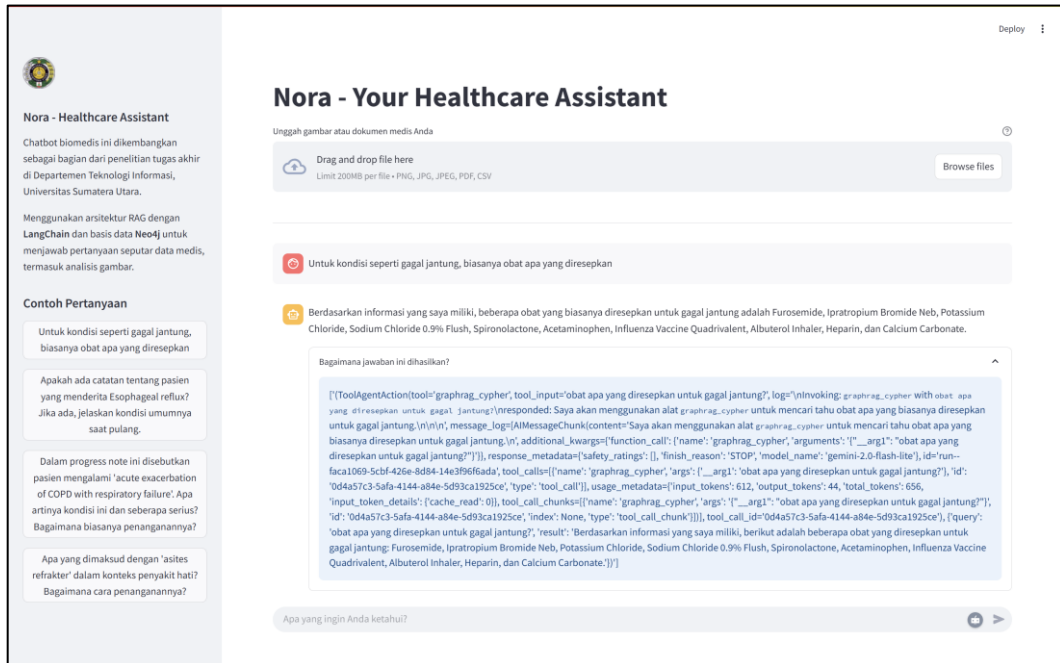
Tabel 4.4 Analisis Akurasi Routing Tool oleh *Agent*

Kategori Pertanyaan	Jumlah Pertanyaan	<i>Tool</i>	Akurasi (%)
Faktual-Graf	20	<i>graphrag_cypher</i>	100
Naratif-Vektor	15	<i>note_reviewer</i>	100
Total Keseluruhan	35		100

4.3.2 Analisis Kualitatif dan Studi Kasus

Evaluasi kuantitatif yang telah disajikan pada sub-bab sebelumnya memberikan gambaran komprehensif mengenai performa sistem secara numerik. Namun, untuk mencapai pemahaman yang mendalam terhadap kekuatan dan limitasi dari arsitektur agent GraphRAG yang diimplementasikan, diperlukan analisis kualitatif melalui studi kasus terhadap interaksi spesifik sistem. Analisis ini dimungkinkan melalui konfigurasi *return_intermediate_steps=True* pada komponen *AgentExecutor*, yang memfasilitasi pelacakan terhadap proses reasoning internal agent selama eksekusi.

A. Studi Kasus : Efektivitas Penelusuran *Knowledge Graph*



Gambar 4.4 Contoh tanya-jawab untuk *GraphRAG Cypher*

```
INFO: 127.0.0.1:57688 - "POST /hospital-multimodal-agent HTTP/1.1" 200 OK
-> No file detected. Invoking RAG agent.

> Entering new AgentExecutor chain...

Invoking: `graphrag_cypher` with `obat apa yang diresepkan untuk gagal jantung?`
responded: Saya akan menggunakan alat `graphrag_cypher` untuk mencari tahu obat apa yang biasanya diresepkan untuk gagal jantung.

{'query': 'obat apa yang diresepkan untuk gagal jantung?', 'result': 'Berdasarkan informasi yang saya miliki, berikut adalah beberapa obat yang diresepkan untuk gagal jantung: Furosemide, Ipratropium Bromide Neb, Potassium Chloride, Sodium Chloride 0.9% Flush, Spironolactone, Acetaminophen, Influenza Vaccine Quadrivalent, Albuterol Inhaler, Heparin, dan Calcium Carbonate.'}Berdasarkan informasi yang saya miliki, beberapa obat yang diresepkan untuk gagal jantung adalah Furosemide, Ipratropium Bromide Neb, Potassium Chloride, Sodium Chloride 0.9% Flush, Spironolactone, Acetaminophen, Influenza Vaccine Quadrivalent, Albuterol Inhaler, Heparin, dan Calcium Carbonate.

> Finished chain.
```

Gambar 4.5 log proses *graphrag_cypher*

Dengan Gambar 4.4 dan 4.5, Studi kasus ini mengevaluasi kemampuan sistem dalam menangani pertanyaan natural berbasis literasi kesehatan yang mengarah pada pemahaman klinis berbasis graf pengetahuan.

- Pertanyaan Evaluasi:

“Obat apa yang diresepkan untuk gagal jantung?”

Analisis Proses *Reasoning Agent (Intermediate Steps)*:

- a) **Fase *Intent Recognition***: *Agent* mengenali bahwa pertanyaan mengandung entitas klinis "gagal jantung" dan kata kerja "diresepkan", yang menandakan kebutuhan untuk menelusuri data medication berdasarkan diagnosis.
 - b) **Fase *Tool Selection***: *Agent* memutuskan untuk mengeksekusi *graphrag_cypher* karena pertanyaan bersifat faktual dan dapat dijawab dengan query terhadap struktur graph medis yang terhubung antar entitas.
 - c) **Fase *Query Formulation***: *Agent* menyusun query untuk mengekstraksi daftar obat berdasarkan diagnosis “gagal jantung” dari node yang terhubung, tanpa mengikat ke ID admission spesifik, sehingga bersifat general dan edukatif.
 - d) **Fase *Data Retrieval & Abstraction***: *Tool graphrag_cypher* mengeksekusi *Cypher Query* dan berhasil mengembalikan data struktural berupa daftar node obat terkait, termasuk *Furosemide*, *Ipratropium Bromide Neb*, *Potassium Chloride*, *Spironolactone*, dan lainnya.
 - e) **Fase *Verbalization***: LLM mengonversi hasil structured graph ke dalam kalimat natural yang dapat dipahami oleh pengguna non-medis, sambil tetap mempertahankan integritas fakta klinis.
- Output Sistem yang Dihasilkan: “Berdasarkan informasi yang saya miliki, beberapa obat yang biasanya diresepkan untuk gagal jantung adalah *Furosemide*, *Ipratropium Bromide Neb*, *Potassium Chloride*, *Sodium Chloride* 0.9% *Flush*, *Spironolactone*, *Acetaminophen*, *Influenza Vaccine Quadrivalent*, *Albuterol Inhaler*, *Heparin*, dan *Calcium Carbonate*.”
- Evaluasi Kinerja: menunjukkan performa yang baik dalam menggabungkan *reasoning* berbasis *graph* dengan penalaran *natural language*. Keberhasilan terletak pada tiga aspek utama: (1) Pemilihan tool berdasarkan semantik

Dari Gambar 4.6 dan 4.7, Studi kasus ini mengevaluasi kinerja sistem dalam memahami dan menjelaskan istilah medis kompleks yang ditemukan dalam catatan medis pasien.

- Pertanyaan Evaluasi: “Dalam progress note ini disebutkan pasien mengalami ‘*acute exacerbation of COPD with respiratory failure*’. Apa artinya kondisi ini dan seberapa serius? Bagaimana biasanya penanganannya?”
- Analisis Proses Reasoning Agent (Intermediate Steps):
 1. **Fase Intent Detection:** Agent mengenali bahwa pertanyaan mengandung istilah klinis kompleks dari catatan medis dan memerlukan klarifikasi istilah serta informasi mengenai gejala dan penanganan.
 2. **Fase Tool Selection:** *Agent* secara tepat melakukan *routing* ke *tool note_reviewer*, karena konteks berasal dari note dan memerlukan *reasoning* berbasis dokumentasi klinis.
 3. **Fase Retrieval melalui RAG:** *Tool note_reviewer* menggunakan $k=2$ dalam parameter pencarian (*search_kwargs*), sehingga mengambil dua dokumen terdekat dari index berbasis *embedding* untuk dijadikan dasar penjawaban.
 4. **Fase Reasoning & Synthesis oleh LLM:** Berdasarkan dua dokumen terambil, sistem menghasilkan penjelasan lengkap yang mencakup:
 - Definisi eksaserbasi akut COPD.
 - Gejala khas (sesak napas, batuk, *wheezing*).
 - Pemeriksaan yang umum dilakukan (laboratorium, *imaging*).
 - Penanganan umum dan intensif (bronkodilator, steroid, ICU).
 - Dua contoh kasus nyata sebagai ilustrasi tingkat keparahan dan pendekatan personalisasi perawatan.
- Output Sistem yang Dihasilkan: “Berdasarkan catatan medis, pasien mengalami eksaserbasi akut COPD (peningkatan gejala COPD secara tiba-tiba) dengan beberapa gejala yang mengindikasikan adanya gangguan pernapasan...”
(dilanjutkan dengan gejala, penanganan, dan contoh kasus seperti pada tangkapan layar).
- Evaluasi Kinerja: Sistem menunjukkan performa *reasoning* yang sangat baik,

dengan keluaran yang edukatif dan berbasis fakta klinis. Keberhasilan sistem dipengaruhi oleh: (1) **Pemilihan tool yang tepat (*note_reviewer*)**, (2) **Kemampuan sistem RAG mengambil dua dokumen yang relevan ($k=2$)**, dan (3) **Kejelasan LLM dalam menyintesis dokumen ke dalam jawaban yang sistematis, menggunakan struktur ‘Definisi–Gejala–Penanganan–Contoh Kasus’**. Hasil ini mendukung kemampuan sistem sebagai asisten kesehatan berbasis literasi medis untuk menjelaskan konten *note* secara kontekstual dan informatif kepada pengguna non-klinis.

4.3.3 Uji Coba dan Analisis Multimodal

Penulis juga mencoba untuk menggunakan kedua *tool*, berikut adalah analisis nya. Berdasarkan eksperimen pertanyaan gabungan dari dua sumber data (*graph* dan *note*) terkait prosedur ***Thoracentesis* pada admisi 20345487**, berikut adalah **analisis studi kasus** yang merefleksikan proses reasoning sistem *GraphRAG multi-tool* dengan pendekatan evaluatif seperti sebelumnya.

- Pertanyaan Evaluasi: “Dengan menggunakan kasus dari admisi 20345487, jelaskan kepada orang awam apa itu prosedur *Thoracentesis* dan mengapa itu dilakukan.”
- Analisis Proses Reasoning Agent (*Multi-tool Inference*)
 1. Fase *Intent Understanding*: Pertanyaan ini mencakup dua fokus utama: (a) penjelasan prosedur medis “*Thoracentesis*” secara umum, dan (b) alasan spesifik prosedur tersebut dilakukan pada **kasus nyata** pasien. Artinya, sistem perlu menggabungkan **pengetahuan umum** (dari *graph* atau base knowledge) dengan **konteks spesifik pasien** (dari catatan medis/*note*).
 2. Fase *Tool Routing*: Agent secara cerdas melakukan **multi-tool reasoning**, yaitu:
 - o Menggunakan **GraphCypherQACChain** untuk memahami relasi entitas Procedure dengan Admission berdasarkan ID 20345487.
 - o Mengaktifkan **NoteReviewer** untuk menggali informasi tambahan dari catatan medis pasien guna menjelaskan gejala, riwayat penyakit, dan hasil prosedur secara deskriptif.
 3. Fase *Retrieval* dan *Reasoning*:

- Dari graph, diperoleh bahwa pasien menjalani prosedur Thoracentesis.
- Dari note, sistem menemukan konteks medis: **pasien memiliki sirosis hati akibat Hepatitis C** yang menyebabkan **hidrotoraks hepatic**, yaitu penumpukan cairan di rongga pleura (paru kanan), dan keluhan utama pasien adalah **sesak napas memburuk**.

4. Fase *Verbalization* oleh LLM: Sistem menyusun jawaban **berstruktur edukatif** yang membagi respons ke dalam dua bagian:

- Penjelasan awam mengenai *apa itu Thoracentesis*, termasuk analogi "balon dalam kotak dada".
- Penjelasan alasan tindakan tersebut dilakukan dalam konteks klinis pasien, lengkap dengan penyebab (sirosis hati), gejala (sesak napas berat), dan hasil prosedur (pengeluaran cairan sebanyak **3,35 liter**).

- Output Sistem:

“Thoracentesis adalah tindakan medis untuk mengeluarkan cairan dari ruang di sekitar paru-paru yang menyebabkan sesak napas. Pada kasus pasien ini, cairan menumpuk akibat komplikasi dari sirosis hati, dan prosedur ini dilakukan untuk meredakan sesak napas yang berat. Sebanyak 3,35 liter cairan berhasil dikeluarkan melalui prosedur ini.”

Dengan skor metrik pada uji coba ini ialah :

Tabel 4.5 Analisis Akurasi Multi Tool - oleh Agent

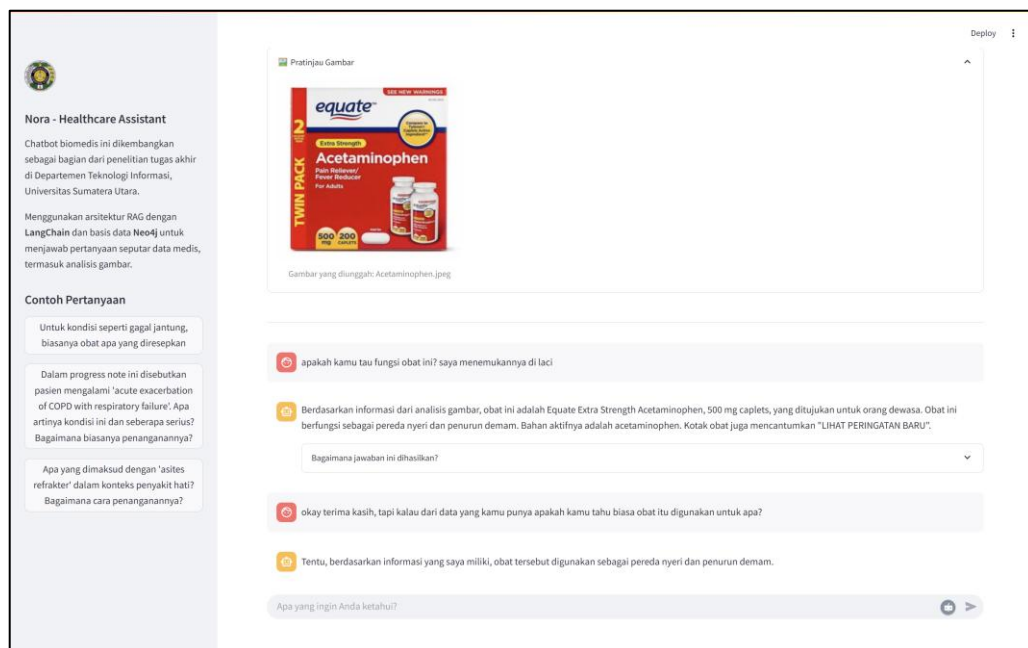
<i>Question</i>	<i>Faithfulness</i>	<i>Answer Relevancy</i>	<i>Context Precision</i>	<i>Context Recall</i>	<i>Answer Correctness</i>
Dengan menggunakan kasus dari admisi 20345487, jelaskan kepada orang awam apa itu prosedur 'Thoracentesis' dan mengapa itu dilakukan.	0.884	0.792	1	1	1

Berdasarkan Tabel 4.5 tersebut, studi ini menunjukkan bahwa sistem berhasil:

- Melakukan **penalaran lintas dua sumber** (*graph + note*).
- Menghasilkan **jawaban klinis yang akurat namun tetap mudah dipahami oleh orang awam**.
- Mampu **mengintegrasikan *reasoning prosedural* dan klinis** dengan cara natural.

Penggunaan pendekatan *multitool* dengan strategi hybrid antara *retrieval structured* dan *unstructured* menunjukkan potensi yang baik untuk pendekatan **GraphRAG multimodal** dalam skenario dunia nyata, terutama untuk *use-case* seperti chatbot **literasi kesehatan berbasis personalisasi medis**. Namun akan sangat dipertimbangkan dari sisi efisiensi token model LLM.

Adapun untuk kasus uji coba tanya – jawab multimodal yang dilakukan ialah berikut.



Gambar 4.8 Contoh tanya-jawab multimodal


```

INFO: 127.0.0.1:60540 - "POST /hospital-multimodal-agent HTTP/1.1" 200 OK
-> File detected: Acetaminophen.jpeg. Starting Multimodal RAG workflow.
--> Step 1: Extracting information from image with Gemini Vision...
--> Step 1 Result: **Clinically Relevant Information:**
* **Active Ingredient:** Acetaminophen
* **Strength:** 500 mg (Extra Strength)
* **Form:** Caplets
* **Indication:** Pain reliever, fever reducer
* **Target Population:** Adults
* **Quantity:** 200 caplets (total, from twin pack of 2x100 count bottles)
* **Anomaly/Feature:** "SEE NEW WARNINGS" prominently displayed, indicating a need to consult updated safety information or contraindications.
--> Step 2: Invoking RAG agent with new prompt: Using the following context from an image analysis: **Clinically Relevant Information:**
* **Active Ingredient:** Acetaminophen
* **Strength:** 500 mg (Extra Strength)
* **Form:** Caplets
* **Indication:** Pain reliever, fever reducer
* **Target Population:** Adults
* **Quantity:** 200 caplets (total, from twin pack of 2x100 count bottles)
* **Anomaly/Feature:** "SEE NEW WARNINGS" prominently displayed, indicating a need to consult updated safety information or contraindications.', please answer this question: 'okay terima kasih, tapi kalau dari data yang kamu punya apakah kamu tahu biasa obat itu digunakan untuk apa?'

> Entering new AgentExecutor chain...
Tentu, berdasarkan informasi yang saya miliki, obat tersebut digunakan sebagai pereda nyeri dan penurun demam.

```

Gambar 4.9 log proses *graphrag_cypher*

Gambar 4.8 dan 4.9 diatas menampilkan contoh tanya jawab pada sistem menunjukkan bagaimana agent multimodal mampu merespons pertanyaan pengguna berbasis gambar obat. Dalam kasus ini, pengguna mengunggah foto kemasan *Equate Acetaminophen 500 mg* dan bertanya mengenai fungsi obat tersebut. Sistem secara otomatis mengekstraksi informasi penting dari gambar seperti nama obat, dosis, indikasi, dan peringatan, lalu menggunakan informasi tersebut untuk menjawab bahwa obat digunakan sebagai pereda nyeri dan penurun demam.

Poin penting dari analisis ini:

- Sistem berhasil menggabungkan data visual dan teks untuk menjawab pertanyaan pengguna.
- Informasi dari gambar diproses dengan model vision (Gemini) lalu dikontekstualisasi dengan RAG.
- Jawaban yang dihasilkan relevan dan sesuai dengan konteks pertanyaan awam.

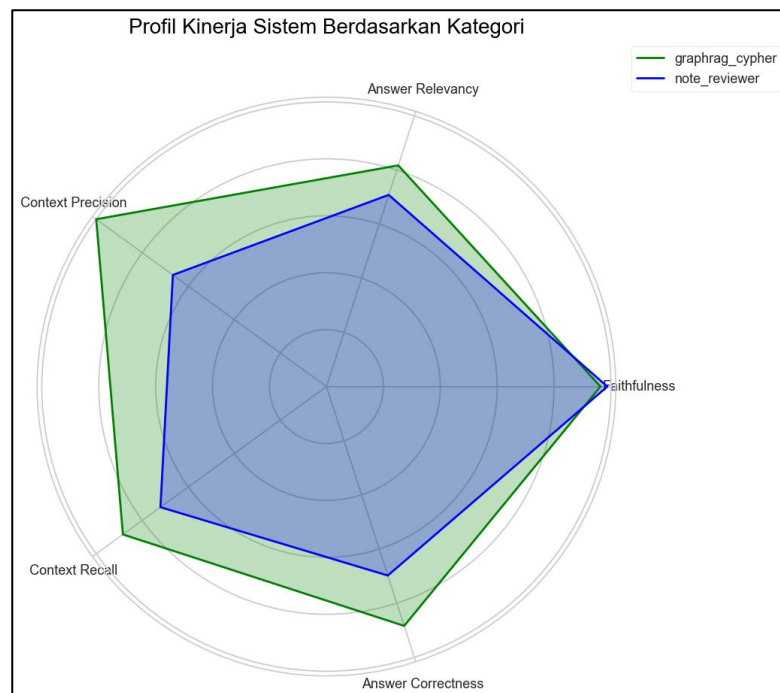
Contoh ini membuktikan bahwa pendekatan multimodal dalam chatbot medis dapat meningkatkan kemampuan sistem dalam memahami input dunia nyata dan memberikan informasi kesehatan yang lebih responsif.

4.4 Diskusi

4.4.1 Interpretasi Hasil Secara Menyeluruh

Analisis menyeluruh terhadap hasil evaluasi menunjukkan bahwa sistem chatbot yang dikembangkan memiliki profil kinerja ganda yang sangat jelas, dengan kekuatan dan tantangan yang spesifik pada setiap modalitas data. Temuan ini tidak hanya memvalidasi desain arsitektur GraphRAG yang diimplementasikan tetapi

juga secara diagnostik memetakan area kompetensinya. Kekuatan utama sistem terletak pada kemampuannya yang superior dalam menangani **data terstruktur**. Hal ini terlihat jelas dari kinerja pada kategori pertanyaan kuantitatif, di mana *tool graphrag_cypher* secara konsisten memberikan skor yang unggul, terutama pada metrik *Context Precision* dan *Answer Correctness*, seperti yang divisualisasikan pada Gambar 4.10. Keberhasilan ini menegaskan bahwa pendekatan menggunakan *knowledge graph* sebagai fondasi basis data sangat efektif untuk pertanyaan faktual.



Gambar 4.10 Visual Radar Kinerja *graphrag_cypher* dan *note_reviewer*

Sebaliknya, **data naratif tidak terstruktur** menjadi tantangan terbesar. Kinerja pada kategori kualitatif, yang bergantung pada *tool note_reviewer*, menunjukkan kelemahan signifikan pada *context precision* dan *context recall*. Hal ini mengindikasikan bahwa meskipun sistem mampu menjaga *Faithfulness* jawaban, kemampuan *retriever* untuk mengekstrak frasa yang paling relevan dari dokumen medis yang padat secara inheren masih terbatas, sebagaimana ditunjukkan oleh area biru yang lebih sempit pada diagram radar (Gambar 4.10).

Di sisi lain, kapabilitas **multimodal (analisis gambar)** menunjukkan potensi yang cukup. Seperti yang ditunjukkan dalam studi kasus, sistem tidak hanya mampu mendeskripsikan konten visual, tetapi juga mensintesis beberapa temuan

radiologis menjadi sebuah inferensi klinis yang relevan. Ini membuka dimensi baru bagi fungsionalitas chatbot, melampaui sekadar interaksi berbasis teks dan graf.

Terakhir, keberhasilan agent dalam mencapai **akurasi 100% pada *tool routing*** adalah temuan fundamental yang positif. Ini membuktikan bahwa lapisan orkestrasi sistem berfungsi dengan andal, memastikan bahwa setiap jenis pertanyaan diarahkan ke *tool* yang paling sesuai untuk dieksekusi.

4.4.2 Implikasi Arsitektur dan Pilihan Teknologi

Keberhasilan *graphrag_cypher* dapat diatribusikan pada sifat basis data graf (Neo4j) yang terstruktur dan bahasa *query Cypher* yang deklaratif. Struktur ini mengurangi ambiguitas, memungkinkan *Large Language Model* (LLM) untuk secara andal menerjemahkan bahasa alami menjadi *query* yang presisi dan deterministik.

Tantangan pada *note_reviewer* menyoroti batasan inheren dari *dense retrieval* berbasis pencarian kemiripan vektor (*semantic search*). Dalam domain biomedis, di mana konteks bisa sangat padat dan nuansa sangat penting, pencarian semantik terkadang hanya mampu mengambil paragraf yang relevan secara topikal, bukan fakta spesifik yang dibutuhkan. Hal ini menyebabkan "*noise*" dalam konteks yang diambil, yang meskipun faktual, tidak semuanya relevan (*low context precision*).

Adapun keberhasilan skenario multimodal sangat bergantung pada kekuatan *Vision Language Model* (VLM) yang digunakan. Kemampuan VLM untuk "menerjemahkan" data visual menjadi deskripsi tekstual adalah kunci. Ini mengubah data yang semula tidak terstruktur (gambar) menjadi konteks terstruktur (deskripsi teks) yang kemudian dapat diproses lebih lanjut oleh *agent*. Ini menunjukkan bahwa untuk arsitektur multimodal yang efektif, kualitas "penerjemah" perantara (VLM untuk gambar, *retriever* untuk teks) adalah sama pentingnya dengan kemampuan *reasoning* LLM itu sendiri.

4.4.3 Refleksi Kritis terhadap Penggunaan RAGAS

Sebagai *framework* evaluasi, RAGAS terbukti menjadi alat yang sangat berharga dalam penelitian ini. Keunggulannya terletak pada kemampuannya untuk

memberikan penilaian kuantitatif yang objektif terhadap berbagai dimensi dari sistem RAG. Metrik *Faithfulness* secara efektif dalam mengukur tingkat halusinasi, sementara *Context Precision* dan *Context Recall* memberikan diagnosis yang jelas terhadap kinerja komponen *retriever*.

Namun, terdapat beberapa batasan yang perlu disadari. Pertama, metrik *Answer Correctness* sangat bergantung pada kualitas dan keunikan *reference_answer* dalam *golden dataset*. Untuk pertanyaan yang bersifat terbuka atau kualitatif (misalnya, rangkuman medis), **bisa terdapat beberapa versi jawaban yang sama-sama benar, yang mungkin tidak dapat dinilai secara akurat oleh perbandingan semantik otomatis**. Kedua, RAGAS tidak dirancang untuk mengevaluasi kualitas interpretasi visual secara langsung; ia hanya dapat mengevaluasi jawaban tekstual yang dihasilkan *setelah* gambar diinterpretasikan. Oleh karena itu, penilaian pada skenario multimodal masih memerlukan validasi kualitatif oleh manusia.

4.4.4 Limitasi Penelitian dan Arah Pengembangan Selanjutnya

Penelitian ini memiliki beberapa batasan yang membuka peluang untuk pengembangan di masa depan.

- **Limitasi Dataset:** *Golden dataset* yang digunakan untuk evaluasi kuantitatif terdiri dari 35 pertanyaan. Meskipun mencakup tiga kategori, jumlah ini masih relatif kecil dan mungkin belum merepresentasikan seluruh spektrum kompleksitas pertanyaan dalam domain biomedis.
- **Limitasi Evaluasi:** Evaluasi dilakukan secara otomatis (RAGAS) dan kualitatif oleh peneliti. Belum ada validasi atau studi pengguna yang melibatkan ahli domain seperti dokter atau radiolog untuk menilai kegunaan klinis dan keakuratan sistem dari perspektif praktisi.
- **Limitasi Generalisasi:** Sistem dikembangkan dan diuji menggunakan dataset MIMIC-IV. Performanya pada jenis rekam medis elektronik (EMR) dari institusi lain yang mungkin memiliki format dan terminologi berbeda belum diketahui. Berdasarkan temuan dan limitasi tersebut, arah pengembangan selanjutnya dapat difokuskan pada:

1. **Peningkatan Komponen Retriever:** Mengimplementasikan teknik *re-*

ranking pada hasil pencarian semantik untuk meningkatkan Context Precision pada data naratif.

2. **Studi Pengguna dengan Ahli Domain:** Melakukan sesi evaluasi interaktif dengan dokter atau mahasiswa kedokteran untuk mendapatkan umpan balik kualitatif mengenai kejelasan, keakuratan, dan kegunaan chatbot dalam alur kerja klinis.
3. **Ekspansi Kemampuan Multimodal:** Menguji dan mengintegrasikan model VLM yang lebih terspesialisasi untuk domain medis untuk meningkatkan akurasi analisis gambar, serta mengeksplorasi modalitas lain seperti data sinyal (misalnya, EKG).
4. ***Fine-tuning* LLM:** Melakukan *fine-tuning* pada model bahasa yang digunakan untuk lebih memahami terminologi medis spesifik dan konteks dari dataset MIMIC-IV, yang berpotensi meningkatkan kinerja di semua skenario. (Gupta dkk., 2022; Jiang, 2024; San-Juan dkk., 2024)

BAB 5

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil penelitian dan evaluasi yang telah dilakukan terhadap chatbot biomedis berbasis *Graph-Based Retrieval-Augmented Generation* (GraphRAG), dapat ditarik beberapa kesimpulan utama sebagai berikut:

1. Integrasi GraphRAG Berhasil Meningkatkan Kualitas Jawaban:

Implementasi arsitektur GraphRAG yang menggabungkan *knowledge graph* (Neo4j), *Large Language Model* (Google Gemini 2.5 Pro), dan kerangka kerja LangChain terbukti berhasil mengatasi tantangan fundamental pada LLM konvensional, yaitu *hallucination* dan ketidakakuratan faktual. Sistem mampu memberikan jawaban yang tidak hanya relevan, tetapi juga dapat diverifikasi secara faktual berdasarkan data sumber dari dataset MIMIC-IV v3.1.

2. Kinerja Spesialisasi Tool yang baik: Sistem menunjukkan performa tinggi pada dua skenario penggunaan yang berbeda. Untuk pertanyaan faktual (*graphrag_cypher*), sistem mencapai *answer_correctness* **88.4%** dengan presisi konteks yang sempurna. Untuk pertanyaan kualitatif berbasis narasi (*review_chain*), sistem mampu mempertahankan *answer_correctness* **69.8%** berkat kemampuan *retrieval* yang komprehensif (*context_recall* 70.8%). Ini membuktikan efektivitas pendekatan agent yang mampu mendelegasikan tugas ke *tool* yang tepat.

3. Akurasi Routing Agent yang Sempurna: Agent yang dirancang dengan *prompt engineering* yang cermat berhasil mencapai akurasi routing 100%. Ini menunjukkan bahwa agent mampu secara otonom dan akurat memilih alat (*graphrag_cypher* atau *note_reviewer*) yang paling sesuai dengan karakteristik pertanyaan pengguna, yang menjadi kunci dari arsitektur hibrid yang responsif dan andal.

4. Potensi sebagai Alat Bantu yang Andal: Dengan skor metrik RAGAS yang solid secara keseluruhan—termasuk *answer_correctness* 80.5%, *faithfulness* 97%, dan *answer_relevancy* 76.9%—chatbot yang dikembangkan menunjukkan potensi cukup besar sebagai alat bantu yang kredibel bagi tenaga medis maupun

pasien. Kemampuan sistem untuk melacak sumber informasi melalui fitur transparansi agent semakin meningkatkan keandalannya.

5.2 Saran

Meskipun penelitian ini telah mencapai tujuannya, terdapat beberapa area yang dapat dikembangkan lebih lanjut untuk meningkatkan kapabilitas dan jangkauan sistem di masa depan. Berikut adalah beberapa saran untuk penelitian selanjutnya:

1. **Adaptasi dan pada Konteks Data Lokal** : Mengadaptasi sistem untuk menggunakan data rekam medis dari rumah sakit di Indonesia. Langkah ini krusial untuk meningkatkan validitas eksternal dan relevansi klinis, memastikan model sesuai dengan karakteristik demografi dan pola penyakit lokal. Penelitian masa depan dapat merintis kolaborasi untuk membangun atau memanfaatkan dataset klinis nasional yang ter-deidentifikasi.
2. **Perkaya *Knowledge Graph* dengan Data Heterogen** : Mengintegrasikan sumber pengetahuan eksternal yang beragam seperti pedoman klinis nasional, jurnal medis (PubMed), dan data farmasi. Ini akan membangun *knowledge graph* yang lebih komprehensif dan meningkatkan kemampuan penalaran chatbot.
3. **Implementasi Penuh Kapabilitas Multimodal** : Mengembangkan kemampuan analisis multimodal secara penuh, tidak hanya mengunggah file. Ini melibatkan implementasi model *vision-language* untuk mengekstraksi informasi secara langsung dari citra medis (misalnya, X-ray, CT Scan) dan dokumen terstruktur (PDF), menjadikan interaksi dengan chatbot lebih kaya dan intuitif.
4. **Optimalisasi Presisi Konteks dengan Teknik Lanjutan** Meningkatkan metrik *context precision* (saat ini 66,7% pada pencarian naratif) dengan mengeksplorasi teknik *text chunking* dinamis dan algoritma *re-ranking*. Tujuannya adalah untuk menyajikan segmen informasi yang paling relevan kepada LLM.
5. **Evaluasi Holistik**: Dari Pengguna hingga Uji Klinis, melakukan evaluasi sistem secara menyeluruh yang melampaui metrik teknis (RAGAS). Ini mencakup evaluasi *User Experience* (UX) bersama praktisi klinis untuk menyempurnakan alur kerja, dan dilanjutkan dengan uji klinis terkontrol untuk memvalidasi efektivitas, keamanan, dan dampak nyata sistem di lingkungan perawatan kesehatan.

DAFTAR PUSTAKA

- Abbasian, M., Azimi, I., Rahmani, A. M., & Jain, R. (2023). *Conversational Health Agents: A Personalized LLM-Powered Agent Framework*. <http://arxiv.org/abs/2310.02374>
- Baltrusaitis, T., Ahuja, C., & Morency, L.-P. (2019). Multimodal Machine Learning: A Survey and Taxonomy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(2), 423–443. <https://doi.org/10.1109/TPAMI.2018.2798607>
- Barrasa, J., & Webber, J. (2023). *Building Knowledge Graphs A Practioner's Guide* (First). O'Reilly Media, Inc.
- Besta, M., Gerstenberger, R., Peter, E., Fischer, M., Podstawski, M., Barthels, C., Alonso, G., & Hoefler, T. (2024). Demystifying Graph Databases: Analysis and Taxonomy of Data Organization, System Designs, and Graph Queries. *ACM Computing Surveys*, 56(2), 1–40. <https://doi.org/10.1145/3604932>
- Bhayana, R. (2024). Chatbots and Large Language Models in Radiology: A Practical Primer for Clinical and Research Applications. *Radiology*, 310(1), e232756. <https://doi.org/10.1148/radiol.232756>
- Chang, Y., Wang, X., Wang, J., Wu, Y., Yang, L., Zhu, K., Chen, H., Yi, X., Wang, C., Wang, Y., Ye, W., Zhang, Y., Chang, Y., Yu, P. S., Yang, Q., & Xie, X. (2023). *A Survey on Evaluation of Large Language Models*. <http://arxiv.org/abs/2307.03109>
- Delile, J., Mukherjee, S., Van Pamel, A., & Zhukov, L. (2024). *Graph-Based Retriever Captures the Long Tail of Biomedical Knowledge*. <http://arxiv.org/abs/2402.12352>
- Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation*.
- Fernandes, D., & Bernardino, J. (2018). Graph Databases Comparison: AllegroGraph, ArangoDB, InfiniteGraph, Neo4J, and OrientDB. *Proceedings of the 7th International Conference on Data Science, Technology and Applications*, 373–380. <https://doi.org/10.5220/0006910203730380>
- Ge, J., Sun, S., Owens, J., Galvez, V., Gologorskaya, O., Lai, J. C., Pletcher, M. J., & Lai, K. (2023). Development of a Liver Disease-Specific Large Language Model Chat Interface using Retrieval Augmented Generation. *medRxiv : the*